

The Alberta Buck - Ethereum Implementation (DRAFT v0.6)

Perry Kundert

2026-04-20 12:00:00



This document presents a concrete Ethereum implementation of the Alberta Buck Architecture, specifying the five core smart contracts:

1. **BUCK_CREDIT** (ERC-721): An NFT representing an insurer's offer of parametric insurance on a real-world asset, with deterministic depreciation curves and piecemeal client activation.
2. **IdentityRegistry** (custom): An on-chain registry of per-address Pointcheval-Sanders signature credentials and ElGamal-encrypted identity points, verified via a BN254-precompile NIZK at registration time. This is the trust anchor every BUCK transfer is gated on – see the companion Identity and Proofs documents for the full cryptographic design.
3. **BUCK** (ERC-20): A fungible token minted against aggregated BUCK_CREDIT values, with on-chain credit-limit enforcement, default insurance premiums, and **identity-aware approve** and **transfer** flows that enforce the BUCK mutual-decryptability invariant on every counterparty pair via Chaum-Pedersen NIZK re-encryption proofs.
4. **Notes** (custom): A privacy-preserving denomination layer on top of BUCK. Clients escrow BUCK into a commitment pool by posting a Groth16 SNARK that ties the deposited **totalFace** to a batch of Poseidon-hash note commitments; later, any holder can redeem a single note by presenting a second Groth16 SNARK proving membership in the on-chain Tornado-style incremental Poseidon Merkle tree under a fresh nullifier. Redemption pays out via **BUCK.transferCarrying**, which weighted-merges the recipient's demurrage age with the pool's so the SNARK's **face** $===$ **v** value-conservation invariant holds at the ERC-20 boundary. See Notes and Proofs for the circuit design.

5. **BUCK_K** (custom): A controller that supplies the dynamic Value Stabilization Factor used in credit-limit computation. The first deployment uses `BuckKControllerStatic`, a governance-settable constant, so the identity and notes layers can land first; the eventual on-chain PID + commodity-basket oracle implementation (`BuckKController`) is fully specified below and slots in at the same interface.

This is the first of several planned implementation approaches. The ERC-721 approach is chosen for its clarity, wide tooling support, and natural fit for unique insured assets. Subsequent documents will examine ERC-1155 (semi-fungible batch credits), ERC-3643 (regulated security token with identity compliance), and ERC-4626 (vault-based credit pooling). The **Implementation Plan** at the end of this document specifies the concrete phased build order and tracks completion status.

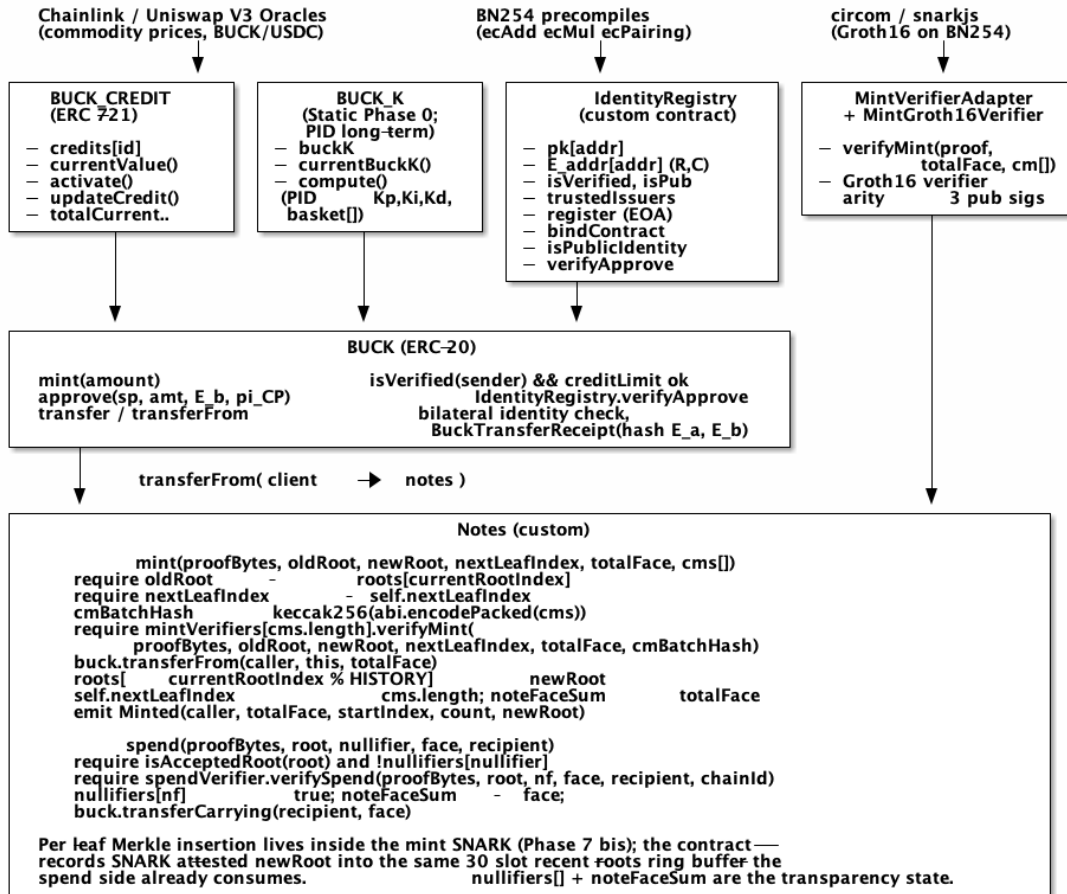
Contents

1	Contract Architecture Overview	5
2	ERC Standard Analysis for BUCK_CREDIT	6
2.1	Recommendation	7
3	BUCK_CREDIT: ERC-721 Insured Asset NFT	7
3.1	Data Model	7
3.2	Depreciation Models	7
3.2.1	Depreciation Examples	8
3.3	Activation (Execution) Model	8
3.4	Insurer Updates	8
4	IdentityRegistry: On-Chain Trust Anchor	9
4.1	Cryptographic Surface	9
4.2	Solidity Surface	9
4.3	Public-Identity Contracts	11
4.4	Issuer Trust	11
5	BUCK: ERC-20 Token	11
5.1	mint() Implementation	11
5.2	Identity-Aware approve()	12
5.3	Identity-Aware transfer() and transferFrom()	12
5.4	Demurrage and transferCarrying()	13
5.5	Default Insurance Premium	13
6	BUCK_K: Value Stabilization Controller	14
6.1	Process Variable and Setpoint	14
6.2	Solidity Implementation	14
6.3	PID Tuning Considerations	15
6.4	Adaptive Gain Scheduling (Future Enhancement)	15
7	Notes: Privacy-Preserving Denomination	16
7.1	Mint Circuit (circom, Groth16 on BN254)	16
7.2	Spend Circuit (circom, Groth16 on BN254)	17
7.3	Verifier Interfaces and Adapters	17
7.4	Notes Contract	18
8	Integration: Full mint() and transfer() Walkthrough	19
8.1	Step 0: Identity Registration (One-Time)	19
8.2	Step 1: Mint	19
8.3	Step 2: Approve Bob (Identity Handshake)	20
8.4	Step 3: Bob Pulls the Transfer	20
8.5	Step 4: Alice Denominates 125 BUCK as Notes	21
8.6	Step 5: Alice (or her recipient) Spends a Note	21
9	Gas Cost Analysis	22

10 Security Considerations	23
10.1 Oracle Manipulation	23
10.2 Credit Enumeration DoS	23
10.3 Insurer Trust	23
10.4 Reentrancy	23
11 Future Approaches	23
11.1 ERC-1155: Semi-Fungible Batch Credits	24
11.2 ERC-3643 (T-REX): Regulated Security Token	24
11.3 ERC-4626: Vault-Based Insurance Pool	24
11.4 L2 and Cross-Chain Deployment	24
12 Implementation Plan	24
12.1 Phase Completion Status (current)	25
12.2 Cryptographic Correctness Review of Companion Memos	25
12.2.1 alberta-buck-notes-rollup-mint.org – Phase 7-bis batch-mint design	25
12.2.2 alberta-buck-notes-serialized.org – Serialized B1 sub-notes design	26
12.3 Testing Strategy: Three Layers	27
12.4 Off-Chain Wallet (alberta_buck.wallet, Python)	28
12.5 Phase 0: Stub the Surrounding Contracts	28
12.6 Phase 1: BN254 Helper Library	29
12.7 Phase 2: IdentityRegistry Contract	29
12.8 Phase 2-A: Simulated Issuer (Government Registry) + Test-Credential Pipeline	29
12.9 Phase 3: Identity-Aware Buck ERC-20	30
12.10Phase 4: Public-Identity Contract Support	31
12.11Phase 5: Notes Scaffolding	31
12.12Phase 6: Groth16 Mint Circuit + Verifier	32
12.13Phase 7: Spend Circuit + Nullifier Reveal	33
12.13.1 Resolved Items from the Notes Correctness Review	34
12.14Phase 7-bis: Batch-Mint Pivot (in-circuit Merkle insertion)	35
12.15Phase 8: A-flavor Spend (spend_a.circom V2 + off-chain CP-DLEQ) – Shipped	36
12.15.1 V1: spend_a.circom predicate-only A-spend	36
12.15.2 V2: In-circuit Poseidon-8 binding + off-chain CP-DLEQ + Solidity precom- pile verifier	36
12.16Phase 9: Serialized B1 Sub-Notes (spend_serialized.circom + per-parent bitmap)	38
12.17Phase 10: Live BuckKController (PID + AMM-pool oracles)	39
12.18Phase 11: BUCK Liquidity Bootstrap + BUCK/USDC AMM Pool	40
12.19Phase 12: BUCK_CREDIT Insurance Integration in Buck.mint	41
12.20Phase 13: Production MPC Trusted-Setup Ceremony	42
12.21Sequencing and Stop Conditions	42

1 Contract Architecture Overview

The Alberta Buck Ethereum implementation comprises five interacting contracts. The `IdentityRegistry` is the trust anchor of every BUCK transfer: `mint`, `approve` and `transfer` are all gated on the sender (and, where relevant, the recipient) holding a verified, registered identity credential. The `Notes` contract sits on top of BUCK as a Public-Identity counterparty (bound via `IdentityRegistry.bindContract(notes, pk, E, true)`) whose commitments pool is extended only when the caller supplies a Groth16 SNARK binding the deposited `totalFace` to a batch of Poseidon note commitments. Every account *and* contract that operates on BUCKs is required to carry an Identity binding – EOAs via `register`, contracts via `bindContract` – so that bilateral identity-checked `transfer` / `transferFrom` calls have a verifiable counterparty on both sides.



The flow is:

1. An **insurer** creates a `BUCK_CREDIT` NFT representing their offer to insure a client's asset.
2. The **client** (asset owner) activates all or part of their credit, paying the current premium.
3. Before the client can interact with BUCK at all, they **register** a Pointcheval-Sanders identity credential and ElGamal-encrypted identity point with `IdentityRegistry`. The contract verifies the issuer's PS signature and the binding NIZK proof on-chain via the BN254 pairing precompile.

4. When the client calls `BUCK.mint()`, the contract requires `IdentityRegistry.isVerified(sender)`, then aggregates the client’s active credits, applies depreciation, multiplies by the current `BUCK_K`, and mints BUCKs up to the resulting limit.
5. To send BUCKs, the sender first calls `BUCK.approve(spender, amount, E_bob, pi_CP)`, which routes the recipient’s re-encrypted identity ciphertext and Chaum-Pedersen proof through the `IdentityRegistry`; the proof binds `E_bob` to the sender’s registered credential without ever revealing the underlying identity point M .
6. `transfer` / `transferFrom` then perform a bilateral identity check: both parties must be registered (`isVerified`). Per-pair receipt fragments stored at `approve` time are the primary identity material in the receipt; if no fragment exists and at least one party is bound under a Public Identity (`isPublicIdentity`), the bound `_identityHash` falls in as the fragment for that side. EOA $\leftrightarrow$ EOA Encrypted-Identity transfers MUST have a prior CP receipt. The contract emits a `BuckTransferReceipt` event carrying only ciphertext hashes.
7. To denominate holdings as private notes, the client calls `Notes.mint(proofBytes, cm[], totalFace)`. The `MintVerifierAdapter` decodes the proof as (`uint256[2]`, `uint256[2][2]`, `uint256[2]`) and forwards to the snarkjs-generated `MintGroth16Verifier`, which checks a Groth16 proof with public signals [`totalFace`, `cm[0]`, `cm[1]`]. The circuit enforces $\text{totalFace} = \sum v_i$ and $\text{cm_i} = \text{Poseidon}(\text{flavor_i}, v_i, \text{rho_i}, \text{idHash_i}, \text{predicate_i})$ per note. The `Notes` contract was bound under a Public Identity at deployment via `IdentityRegistry.bindContract(notes, pk, E, true)` (or via `BuckAwareDeployer.deployAndBind` for atomic deploy+bind), so its `transferFrom` pull from the minter satisfies the identity-aware BUCK transfer rules: both parties are `isVerified`, and the Notes-side receipt fragment falls back to the deterministic `_identityHash` because the pool’s operator has publicly disclosed m off-chain.
8. `BUCK_K` ships in two stages. **Phase 0** uses `BuckKControllerStatic` with a governance-settable value, so the identity and notes layers can land first. The on-chain PID + commodity-oracle implementation (`BuckKController`, described in the `BUCK_K` section) is the eventual replacement.

2 ERC Standard Analysis for BUCK_CREDIT

Before diving into the ERC-721 implementation, we compare candidate standards against the BUCK_CREDIT requirements:

Requirement	ERC-721	ERC-1155	ERC-3643 (T-Rex)	ERC-721
Unique per-asset credit	Native	Possible (NFT mode)	Native	Poor
Insurer-mutable parameters	Custom role	Custom role	Built-in agent roles	Not c
Deterministic on-chain valuation	Custom view	Custom view	Custom view	share
Piecemeal client activation	Custom state	Fungible sub-amounts	Custom state	depos
Transfer restrictions	Custom	Custom	Built-in compliance	Open
Enumerable (for mint aggregation)	ERC721Enumerable	balanceOf per ID	Inherits ERC-20	Singl
Regulatory identity / KYC	Not built-in	Not built-in	ONCHAINID native	Not b
Tooling / ecosystem maturity	Excellent	Very good	Moderate (niche)	Exce

2.1 Recommendation

ERC-721 is the strongest first choice:

- Each **BUCK_CREDIT** is genuinely unique (specific asset, depreciation curve, insured value, premium schedule). This is precisely what **ERC-721** models.
- The **ERC721Enumerable** extension provides `tokenOfOwnerByIndex()`, required for `BUCK.mint()` to aggregate all credits in an account.
- Custom extensions for dual-role access (owner activates, insurer updates) and deterministic valuation are straightforward.
- Widest tooling support, most audited implementations, lowest learning curve.

ERC-3643 is the strongest future candidate, especially if regulatory compliance (KYC, transfer restrictions, identity registry) becomes a deployment requirement. Its agent-role system maps naturally to the insurer/client relationship. However, its dependency chain (**ONCHAINID**, compliance modules, claim topics) adds significant complexity that is better deferred.

ERC-1155 is attractive if standardized credit "classes" emerge (e.g., all Calgary residential properties with the same depreciation model), allowing semi-fungible batching. For the initial system where each credit is individually appraised, the fungibility benefit doesn't apply.

ERC-4626 is a poor fit for individual credits but may be useful for the **InsurancePool** (where mutual insurance shares *are* fungible vault interests).

3 BUCK_CREDIT: ERC-721 Insured Asset NFT

3.1 Data Model

Each **BUCK_CREDIT** token is an **ERC721Enumerable** whose per-token **CreditParams** struct (`src/BuckCredit.sol`) carries all the insurer's current offer and the client's activation state. All monetary values are 18-decimal **BUCK**-equivalent units. Fields group naturally into three zones of mutability:

- **Immutable** at mint: **insurer** (the vendor address that may update this token), **assetClass**, **createdAt**.
- **Insurer-mutable** via `updateCredit`: **faceValue**, **depreciationFloor**, **depType**, **depRate**, **depStartAt**, **premiumRate**, **lastUpdated**.
- **Client-mutable** via `activate`: **activatedValue** ($\leq$ **faceValue**), **lastActivatedAt**.

DepreciationType is one of **NONE** (land, gold), **LINEAR** (constant annual reduction), or **DECLINING_BALANCE** (percentage of remaining value). Three events – **CreditCreated**, **CreditUpdated**, **CreditActivated** – record every state change. The per-token **insurer** field lets a single deployment host many insurers; **assetClass** cannot be reclassified after mint.

3.2 Depreciation Models

Depreciation is computed deterministically from the NFT's on-chain parameters in `currentValue(tokenId)` – a pure view with no oracle input, gas-free to query. The model branches on **depType**:

- **NONE**: **currentValue** is the activated face value, unchanged.

- **LINEAR**: loss accrues at `depRate` basis points per year on the depreciable portion (`faceValue - floor`), clamped at the `depreciationFloor`.
- **DECLINING_BALANCE**: continuous-compounded version using an in-contract `_expNeg` (6th-order Taylor series, accurate to $<0.01\%$ for $x < 3.0$ – i.e. up to $\sim 300\%$ cumulative depreciation). A production deployment should swap in `PRBMath.exp()` or `ABDKMath64x64` for wider-range precision.

In every case the activated portion depreciates proportionally: `currentValue = depreciatedFace * activatedValue / faceValue`.

3.2.1 Depreciation Examples

Asset	Type	Rate	Floor	30yr value (of \$100K)
Land	NONE	–	–	\$100,000
House (bldg)	LINEAR	200bp	20%	\$52,000
Vehicle	DECLINING_BALANCE	1500bp	5%	\$5,633
Farm equip	DECLINING_BALANCE	1000bp	10%	\$16,791
Gold in vault	NONE	–	–	\$100,000

A residential property would typically have *two* BUCK_CREDITS: one for the land (NONE, full face value indefinitely) and one for the structure (LINEAR or DECLINING_BALANCE, with a floor representing salvage/lot-clearing value).

3.3 Activation (Execution) Model

The client executes (activates) their credit piecemeal through `activate(tokenId, amount)` – gated on `ownerOf(tokenId) == msg.sender` and capped at the token’s `faceValue`. Activation increases the `activatedValue` and stamps `lastActivatedAt`; the *premium* for the newly activated amount is collected by `BUCK.mint()`, not here.

`totalCurrentValue(account)` walks `ERC721Enumerable.tokenOfOwnerByIndex` and sums `currentValue(tokenId)` across all tokens owned by `account` – this is the single call `BUCK.mint()` makes to compute the credit limit.

The activation model separates the *decision to use credit* from the *act of minting BUCKs*. A client can activate \$200K of their \$500K home credit today, and months later activate \$100K more – possibly at a different premium rate if the insurer has updated the NFT in the interim.

3.4 Insurer Updates

The insurer can reappraise the asset, change depreciation parameters, and adjust premium schedules. These changes affect *future* credit limit computations, including already-activated amounts (since the depreciation function is computed from current parameters, not historical ones).

`updateCredit(tokenId, newFaceValue, newDepreciationFloor, newDepType, newDepRate, newDepStartAt, newPremiumRate)` is gated on `msg.sender == credits[tokenId].insurer`. If `newFaceValue < activatedValue` the contract caps `activatedValue` downward in the same call; `assetClass` and `insurer` are never changed.

Note that if the insurer reduces `faceValue` below the current `activatedValue`, the activated amount is capped downward. This could push the client’s outstanding BUCK balance above their credit limit, triggering the default insurance mechanism described below.

4 IdentityRegistry: On-Chain Trust Anchor

The `IdentityRegistry` is the binding between Ethereum addresses and the privacy-preserving identity machinery described in the companion documents `Identity with Anonymity`, `Identity Example – Data Flow`, and `Cryptographic Proofs`. Every BUCK `mint`, `approve` and `transfer` consults this registry; an unregistered address cannot hold or move BUCKs.

The registry stores four pieces of state per address:

Field	Type	Source
<code>pk[a]</code>	<code>G_1 point</code>	Identity public key generated by the wallet (EOA) or by the contract operator
<code>E_addr[a]</code>	<code>(R, C) in G_1^2</code>	ElGamal encryption of $M = m \cdot G$ under <code>pk</code> (or <code>(g_1, g_1)</code> for contracts)
<code>isVerified</code>	<code>bool</code>	True after PS + NIZK proofs verify on-chain (EOA) or after <code>bindContract</code> (contract)
<code>isPublicIdentity</code>	<code>bool</code>	True only for contracts bound with the public-disclosure flag via <code>bindContract</code>

It also holds a small set of trusted issuer public keys `trustedIssuers[issuerAddr] = (X, Y)` in G_2^2 , populated by governance. These are the only PS public keys whose signatures the registry will accept at `register()` time.

EOAs always self-register via `register`; the `isPublicIdentity` flag stays `false` for them (an EOA always carries an Encrypted Identity). Contracts bind via `bindContract`, which has no PS / NIZK requirement – trust derives either from atomic `deploy+bind` via `BuckAwareDeployer`, or from first-binder-wins for already-deployed contracts. The flag captures whether the contract operator has chosen to publicly disclose `m` off-chain (typical for AMM pools and other BUCK-unaware contracts whose operator wants the on-chain audit trail to be openable on subpoena).

4.1 Cryptographic Surface

The registry implements three on-chain verifications, all reducible to BN254 precompile calls:

1. **PS signature verification at registration:** $e(\text{sigma}'_1, X + m \cdot Y) = e(\text{sigma}'_2, g_2)$, where the verifier reconstructs the relation linearly in `m` via the binding NIZK in step 2. Costs $\sim 147K$ gas (3-4 `ecPairing` pairs).
2. **NIZK proof binding the rerandomized PS signature to the ElGamal ciphertext.** A Schnorr-family sigma protocol with three response scalars and a Fiat-Shamir challenge bound to `(sigma', E_new, pk_new, registrant_address)`. Costs $\sim 30K$ gas of `ecMul/=ecAdd` on top of the pairing check. Total registration cost is $\sim 235K$ gas (one-time).
3. **Chaum-Pedersen proof of re-encryption correctness at approve time.** The recipient identity ciphertext `E_bob` and proof `pi_CP = (e, s_1, s_2)` arrive in `approve()` calldata; the registry reads `E_alice` from storage (anchoring the proof to the registered credential) and verifies via 3-4 `ecMul` + 3-4 `ecAdd` operations and a Fiat-Shamir reconstruction. Costs $\sim 29K$ gas per counterparty pair.

4.2 Solidity Surface

`src/IdentityRegistry.sol` uses the BN254 helper library for all curve math. The shape of the interface is what matters for the rest of this document:

```

struct G1Point { uint256 X; uint256 Y; }
struct G2Point { uint256[2] X; uint256[2] Y; }

struct ElGamalCT { G1Point R; G1Point C; } // (R, C) = (r*G, r*pk + M)
struct PSSig { G1Point s1; G1Point s2; } // rerandomized PS signature
struct PSPubKey { G2Point X; G2Point Y; } // issuer (X, Y) in G_2

struct RegistrationProof { uint256 e, s_m, s_r; G1Point A_ps, T_C, T_R; }
struct CPProof { uint256 e, s1, s2; G1Point T1, T2, T3; }

function register(address issuer, G1Point pk, ElGamalCT E, PSSig sigma, RegistrationProof proof)

// Bind a (pk, E_addr) Identity to a deployed contract address (target.code.length > 0).
// First binder wins for pre-existing contracts; for atomic deploy+bind use
// BuckAwareDeployer.deployAndBind. isPublicIdentity_=true means the operator
// has chosen to publicly disclose m off-chain.
function bindContract(address target, G1Point pk, ElGamalCT E, bool isPublicIdentity_);

function trustIssuer(address issuer, PSPubKey ipk); // governance
function verifyApprove(address sender, address spender,
                      ElGamalCT E_bob, CPProof pi) view returns (bool);

```

Design notes on the surface:

- Per-address state is `pk`, `E_addr` ((R,C) ciphertext of the identity point M), `isVerified`, `isPublicIdentity`, and `issuerOf` – all public-readable mappings.
- **register** runs three checks in order: (1) recompute the Fiat-Shamir challenge with `msg.sender` binding, (2) verify the Schnorr-family NIZK in `G_1`, (3) verify the PS pairing relation in `G_2`. An already registered / untrusted issuer guard precedes them. Total cost ~235K gas (one-time per address).
- **bindContract** is the only post-registration mutator and is permissioned by the deployment pattern, not by governance: any caller may bind any yet-unbound deployed contract. The intended discipline is operator OPSEC – either the operator is the contract’s deployer and uses `BuckAwareDeployer.deployAndBind` for atomic deploy+bind (no front-running window), or the operator races to bind a pre-existing contract before any other party. Once bound, the binding is immutable.
- **verifyApprove** reads `E_alice` from storage so the caller cannot substitute a fake credential; the `CPProof` structure carries `T1/T2/T3` because the transcript binds `T1` and `T2` separately (only `T3` is recoverable from the verification equations).
- The **RegistrationProof** carries the PS-side commitment `A_ps = m_tilde * sigma'_1` alongside the two ElGamal-side commitments `T_C`, `T_R` – `A_ps` is indispensable for the pairing-product check and cannot be reconstructed from the other responses.
- Governance hooks round out the surface: **revokeIssuer**, **transferGovernance**. Revocation does not un-register existing accounts; they were verified once at registration and are never re-verified.

4.3 Public-Identity Contracts

DeFi infrastructure – AMM pools, vaults, oracles, the Notes pool – cannot generate Chaum-Pedersen proofs at **approve** time because the contract has no private key. Such contracts are bound with `bindContract(target, pk, E, isPublicIdentity_=true)` – the binding has the same `(pk, E_addr)` shape as a self-registered EOA, plus the `isPublicIdentity` flag that records the operator’s commitment to publicly disclose `m` off-chain. Counterparties of a Public-Identity contract still call the identity-bound **approve**, but the bilateral receipt-fragment check on the contract side is satisfied by the deterministic `_identityHash` rather than a per-counterparty CP receipt (see the next section). Public-Identity contracts forfeit content privacy: their identity M is openable on subpoena via the operator’s off-chain attestation, but they retain unforgeability and the audit trail.

A second, deferred mode binds a contract under an Encrypted Identity (`isPublicIdentity_=false`). This is intended for BUCK-aware contracts where the operator runs an off-chain pre-approval flow per counterparty (decrypting each user’s `E_user` to attest that the contract is a willing counterparty), and ships the resulting CP material on-chain so per-user **approve** calls can succeed without divulging the contract’s `m`. The `bindContract` surface supports it; no production contract uses it yet.

4.4 Issuer Trust

`trustedIssuers` is the only governance-controlled trust anchor in the system. Adding a new issuer admits a new origin of registrable identities; removing one stops new registrations under that issuer (existing registrations are unaffected – they were verified at registration time and are never re-verified). Epoch-based credential renewal, key revocation, and issuer rotation are described in the Identity: Epoch-Based Credential Renewal document.

5 BUCK: ERC-20 Token

The BUCK token is an **identity-aware** ERC-20. Beyond the usual mint / transfer / approve surface, every state-changing call routes through the `IdentityRegistry`:

- `mint()` refuses to issue BUCKs to an unverified address.
- `approve()` takes the recipient’s re-encrypted identity ciphertext and a Chaum-Pedersen NIZK proof, and stores a *receipt fragment* binding the (sender, spender) pair to that ciphertext.
- `transfer()` and `transferFrom()` enforce a **bilateral** identity check using the receipt fragments; if a fragment is missing and at least one party is bound under a Public Identity (`isPublicIdentity`), the bound `_identityHash` falls in for that side. The contract then emits a `BuckTransferReceipt` carrying only ciphertext hashes – no plaintext identity ever touches the chain.

This is the on-chain enforcement of the mutual-decryptability invariant: every transfer leaves behind a permanent record from which both counterparties (and only those counterparties, by decrypting their respective ciphertext) can reconstruct the identity of the other.

5.1 `mint()` Implementation

`src/Buck.sol` is constructed with `(buckCredit, buckK, identity, insurancePool).mint(amount)` performs the seven-step sequence:

1. Require `identity.isVerified(msg.sender)`.
2. Read `creditValue = buckCredit.totalCurrentValue(msg.sender)`.
3. Read `k = buckK.currentBuckK()` (Phase-0 static getter or eventual PID `compute()`).
4. `maxLimit = creditValue * k / 1e18`; the stored per-account `storedLimit` ratchets upward only.
5. Refuse if `balanceOf(sender) + amount > storedLimit`.
6. Compute the default-insurance premium (see below).
7. `_mint(sender, amount - premium), _mint(insurancePool, premium), emit Minted(account, amount, premium, creditValue, buckKValue, newLimit)`.

A symmetric `burn(amount)` just calls `_burn(msg.sender, amount)` for repayment.

5.2 Identity-Aware `approve()`

The shipped signature is

```
approve(address spender, uint256 amount, IdentityRegistry.ElGamalCT E_bob, IdentityRegistry.CP pi_CP).
```

The parameterless ERC-20 `approve(address, uint256)` reverts with `BUCK: use identity-bound approve` – the identity-bound form is mandatory. Both `sender` and `spender` must be `isVerified` (i.e. registered EOAs or contracts bound via `bindContract`). Critically, the CP proof is *always* required regardless of the spender’s Identity flavor: even when the spender is a Public-Identity contract, the ERC-20 allowance write must be accompanied by a CP proof. On success the receipt fragment `keccak256(abi.encode(E_bob.R.X, E_bob.R.Y, E_bob.C.X, E_bob.C.Y))` is stored at `_receiptFragments[sender][spender]` and the ERC-20 allowance is written via `_approve(sender, spender, amount)`.

The full rationale for the always-CP rule – consent vs. execution, BN254-key engagement, per-pair freshness, uniform audit semantics, and defense against future identity rotation – is laid out in *Why approve Is Always CP-Bound* in the BUCK Identity document.

For Public-Identity spenders the operator constructs `E_bob` and `pi_CP` by re-encrypting against the spender’s published `m`; for Encrypted-Identity spenders the operator must obtain `E_spender` and pre-approval material from the spender’s wallet (the standard EOA-to-EOA flow).

The `approve` call costs roughly 29K (CP verify) + 46K (standard ERC-20 `approve`) + 22K (receipt `SSTORE`) = ~97K gas, paid once per (sender, spender) pair. Subsequent transfers between the same counterparties incur only the bilateral check and the receipt event (~5K gas of additional work above a baseline ERC-20 transfer).

5.3 Identity-Aware `transfer()` and `transferFrom()`

Both `transfer(to, amount)` and `transferFrom(from, to, amount)` route through `_identityCheckedTransfer`, which enforces:

- `isVerified(from)` and `isVerified(to)` – both parties must carry an Identity binding (EOA via `register`, contract via `bindContract`).

- The **to** side: try `_receiptFragments[from][to]`; if zero, fall back to `_identityHash(to)` *only* if `isPublicIdentity(from) || isPublicIdentity(to)` – otherwise revert with BUCK: **missing identity receipt**. EOA-to-EOA Encrypted transfers **MUST** have a prior approve-time CP receipt; transfers where at least one party is a Public-Identity contract are allowed to use the deterministic fallback.
- The **from** side is best-effort: try the mirrored fragment `_receiptFragments[to][from]` for a reciprocal approve, otherwise fall back unconditionally to `_identityHash(from)`. The receipt still records the asymmetric attestation – auditors reading the event know which side was CP-bound vs. registry-derived.

After the underlying `_transfer` settles, `BuckTransferReceipt(from, to, amount, fromHash, toHash)` is emitted. `_identityHash(account)` is `keccak256(abi.encode(pk.X, pk.Y, R.X, R.Y, C.X, C.Y))` over the account’s stored public key and ElGamal ciphertext – identical for the same account across every transfer it participates in.

The `BuckTransferReceipt` event is the auditable trail: `fromCipherHash` and `toCipherHash` are `keccak256` commitments over the corresponding ElGamal ciphertexts. Counterparties can match a receipt to their off-chain copies of E_{alice}, E_{bob} by recomputing the hash; observers without the underlying ciphertexts learn nothing beyond the fact of a transfer.

5.4 Demurrage and `transferCarrying()`

BUCK carries a continuous on-chain demurrage accrual to drive a perpetual Jubilee schedule. All balance accounting is in *raw* units; the displayed `balanceOf(a)` subtracts a per-account `feeOwing(a)` = `raw_balance(a) * (cumIndexNow - _indexAtLastTouch[a]) / SCALE`. `cumIndex` grows linearly at `BASE_RATE_PER_SEC * (now - genesis)`. Two transfer flavours coexist:

- **Deducting** (default ERC-20 path): the sender’s accumulated fee is burned to the Jubilee recipient’s index slot; sender’s `_indexAtLastTouch` resets to `cumIndex`; the recipient’s index is weighted-merged toward `cumIndex` so its prior fee debt is preserved.
- **Carrying** (`transferCarrying(to, amount)`): no fee burn. `amount` raw units move atomically with the sender’s accrued demurrage age – the recipient’s index becomes the amount-weighted average of its old index and the sender’s `_indexAtLastTouch`: `_indexAtLastTouch[to] = (br * _indexAtLastTouch[to] + amount * idxFrom) / (br + amount)`. The sender’s index is unchanged; sender keeps the residual age on its remaining balance.

`transferCarrying` is the entry point `Notes.spend` and `Notes.spendACP` call when paying out a redeemed face value: it is the reason a recipient absorbs the pool’s average demurrage age (the anonymity cost – and upside, for anyone who held a note longer than the pool average) rather than a discontinuous spike of settled fee. The Jubilee accumulates fee burns into a single sink whose own `_indexAtLastTouch` is itself weighted-merged so disbursements remain age-correct.

5.5 Default Insurance Premium

The premium scales with utilization: the closer the account’s outstanding BUCKs approach the credit limit, the more expensive each additional BUCK becomes. This mirrors the architecture document’s description of Parametric Default Insurance.

The shipped curve is quadratic in utilization: `premium_rate = BASE_RATE + utilization^2 * SCALE_RATE`, with `BASE_RATE = 50bp` (0.50%) and `SCALE_RATE = 450bp` (4.50% additional at

100% utilization). Concretely: $\sim 0.5\%$ at 0% utilization, $\sim 1.75\%$ at 50%, $\sim 4.55\%$ at 90%; 100% is unreachable because the limit check itself rejects. The `_computePremium(account, mintAmount, limit)` helper computes utilization on the post-mint balance and returns the BUCK amount due to the insurance pool.

The quadratic scaling creates a natural disincentive against over-leveraging. A client using 50% of their credit pays roughly 1.75% of each minted BUCK to the insurance pool; at 90% they pay 4.55%. The insurance pool accumulates these premiums to cover defaults (accounts where asset depreciation or BUCK_K changes push outstanding BUCKs above the credit limit).

6 BUCK_K: Value Stabilization Controller

The BUCK_K contract maintains purchasing-power parity between the BUCK and its commodity basket. It ships in two stages behind a shared one-function interface (`currentBuckK()` `external view returns (uint256)`) so the Buck contract is invariant to which stage is deployed:

- **Phase 0 – BuckKControllerStatic:** a governance-settable constant. Constructor (`uint256 _buckK, address _governance`); functions `currentBuckK()`, `compute()` (no-op, returns the stored value), `setBuckK(uint256)` (governance-only), `transferGovernance(address)`. This is the version currently deployed alongside the identity and notes layers so those layers can land first.
- **Phase Later – BuckKController:** a full PID implementation, identical in structure to the project’s ownercredit PID controller, ported to Solidity fixed-point arithmetic. It reads a commodity basket from Chainlink feeds and the BUCK/USDC price from a Uniswap V3 TWAP, runs a P/I/D update on `compute()` once per `dT` seconds, and writes the new `buckK`.

The PID implementation is documented in full below; the Buck ERC-20 reads only `currentBuckK()` from either variant, so swapping one for the other is a constructor argument.

6.1 Process Variable and Setpoint

PID Term	Source	Meaning
Process Variable	BUCK/USDC Uniswap V3 TWAP	What a BUCK actually trades for
Setpoint	Commodity basket oracle sum	What a BUCK <i>should</i> be worth (basket cost)
Error	Setpoint - Process	Positive = BUCK undervalued, negative = overvalued
Output (BUCK_K)	PID computation	Credit limit multiplier (1.0 = neutral)

When BUCK trades *below* basket value (positive error), BUCK_K increases, expanding everyone’s credit limit. This floods the market with cheap BUCKs, which arbitrageurs buy to acquire underpriced commodities, driving the BUCK price back up. The converse reduces BUCK_K when BUCKs are overvalued.

6.2 Solidity Implementation

`src/BuckKController.sol` is a straight Solidity port of the JavaScript PID controller in ownercredit, with 18-decimal fixed-point arithmetic and BN254-friendly types. The shipped shape is:

- **Configuration:** `int256 Kp, Ki, Kd` (gains), `uint256 dT` (minimum update interval), `uint256 buckKMin, buckKMax` (output clamps), `buckUsdcPool, twapInterval, governance`.

- **State:** `int256 P, I, D, uint256 lastUpdate, uint256 buckK` (output, `1e18` = neutral). `basket[]` is an array of `BasketComponent { AggregatorV3Interface feed; uint256 weight; uint8 feedDecimals; }`.
- **Entry point:** `compute()` returns (`uint256`). If `now - lastUpdate < dT` it returns the cached `buckK` without state change. Otherwise it reads `basketCost` (Chainlink-weighted set-point), `buckPrice` (Uniswap V3 TWAP, process variable), runs one PID cycle $\sim \text{rawOutput} = 1e18 + \text{newP} * K_p$
 $- \text{newI} * K_i + \text{newD} * K_d$ (each term divided by `1e18`), clamps to `[buckKMin, buckKMax]` with anti-windup on the integral term, writes the new state, and emits `BuckKUpdated`.
- **View:** `currentBuckK()` view returns (`uint256`) – the Buck contract calls this for non-mutating reads.
- **Governance:** `setGains, setDT, addBasketComponent`.

The TWAP price helper `_getBuckPrice()` uses Uniswap’s `OracleLibrary.consult(pool, twapInterval)` to convert a `tickCumulatives` delta into an 18-decimal price; omitted here because it is mechanical Uniswap V3 boilerplate. The basket helper sums `price * weight` with per-feed decimal normalisation.

6.3 PID Tuning Considerations

The PID gains determine how aggressively BUCK_K responds to price deviations:

Gain	Effect of Increasing	Risk of Over-tuning
Kp	Faster response to price deviations	Oscillation, over-correction
Ki	Eliminates steady-state error (persistent mispricing)	Integral windup, slow recovery
Kd	Dampens oscillation, anticipates trends	Noise amplification

Initial gains should be conservative (high `Kd` relative to `Kp`, low `Ki`) to favor stability over responsiveness. The `dT` parameter (minimum update interval) acts as an additional dampening mechanism – a 1-hour `dT` means BUCK_K changes at most once per hour regardless of how many mints occur, preventing high-frequency oscillation.

The TWAP interval (`twapInterval`) provides manipulation resistance: a 30-minute TWAP means an attacker would need to sustain a price manipulation for 30 minutes to meaningfully affect BUCK_K – a prohibitively expensive proposition on a liquid pool.

6.4 Adaptive Gain Scheduling (Future Enhancement)

The current implementation uses governance-set fixed gains. A future enhancement could implement on-chain gain scheduling where `Kp`, `Ki`, and `Kd` are adjusted based on observed error dynamics:

- **Low volatility regime** (error variance below threshold): reduce `Kp`, increase `Ki` for tighter long-term tracking.
- **High volatility regime** (error variance above threshold): increase `Kp` and `Kd` for faster response, reduce `Ki` to prevent windup.

- **Regime detection:** maintain a rolling variance of the error signal over a window of N updates.

This maps directly to the `PIDController.reset()` pattern in the JavaScript implementation, where gains can be hot-swapped without spiking the derivative term.

7 Notes: Privacy-Preserving Denomination

BUCK transfers carry a public (`from`, `to`, `amount`) quartet. For private value movement, holders denominate their BUCKs as **notes**: a pair of Poseidon-hash commitments posted to the **Notes** contract, backed by an equal amount of BUCK locked in that contract. Notes do not leak values, flavours, or identity bindings – the public state is just a list of opaque 32-byte commitments and a running face-value total.

The design is specified in full in Notes and the underlying SNARK construction in Proofs. This section documents the Ethereum surface: the **Notes** contract, the **IMintVerifier** interface, the circom Groth16 mint circuit, and the adapter that bridges the two.

7.1 Mint Circuit (circom, Groth16 on BN254)

Phase-7-bis ships `circuits/mint_batch.circom`, a parameterised batch-mint circuit that subsumes the per-leaf on-chain Poseidon insertion of Phase 7. One compile per pinned N (recommended {16, 128, 1024}); each pin produces its own `MintGroth16Verifier_N*.sol` and registers in `mintVerifiers[N]` on **Notes**. The public-signals arity is 5 regardless of N.

Signal	Role	Description
<code>oldRoot</code>	public	Live note Merkle root the prover read from chain
<code>newRoot</code>	public	Root after folding <code>cm[0..N-1]</code> starting at <code>nextLeafIndex</code>
<code>nextLeafIndex</code>	public	Live tree size the prover read from chain
<code>totalFace</code>	public	Sum of note values, range-bounded to 128 bits
<code>cmBatchHash</code>	public	<code>keccak256(abi.encodePacked(cm[0..N-1]))</code> – ties calldata to the SNARK
<code>flavor[i]</code>	private	Note flavour (A1/A2/B1 from Notes design)
<code>v[i]</code>	private	Note value (18-decimal integer, range-bounded to 128 bits)
<code>rho[i]</code>	private	Per-note randomness
<code>idHash[i]</code>	private	Hash of the identity binding for this note
<code>predicate[i]</code>	private	Predicate commitment (zero for the plain mint path)
<code>pathSiblings[i][d]</code>	private	Depth-20 sibling hashes used to fold leaf <code>i</code> into the rolling tree

Constraints: N Poseidon-5 opening checks, `Num2Bits(128)` range bounds on each `v[i]` and on `totalFace` (closing the field-overflow attack), one sum check (`totalFace === sum v[i]`), N x `TREE_DEPTH` Poseidon-2 hashes implementing the rolling filled-subtrees insertion from `oldRoot` to `newRoot`, and one keccak-style hash check binding `cmBatchHash` to the per-leaf commitments. Total constraints work out to ~6K R1CS per leaf (~1.7K opening + ~4.3K insertion); see `alberta-buck-notes-rollup` for per-N R1CS / PTAU sizing. `scripts/snark/setup.sh` now runs one phase-2 setup per pinned N, exporting `src/MintGroth16Verifier_N*.sol` for each. `src/SpendGroth16Verifier.sol` is unchanged from Phase 7.

```
interface IMintGroth16 {
    function verifyProof(
```

```

        uint256[2]    calldata a,
        uint256[2][2] calldata b,
        uint256[2]    calldata c,
        uint256[3]    calldata pubSignals    // [totalFace, cm[0], cm[1]]
    ) external view returns (bool);
}

```

The trusted-setup contributions in `setup.sh` use a fixed dev-only entropy string. A production deployment must run a real multi-party ceremony; the same script is the scaffolding.

7.2 Spend Circuit (circom, Groth16 on BN254)

`circuits/spend.circom` proves the redemption of a single B-shape note under an on-chain Merkle root. Public inputs are `[noteRoot, nullifier, face, recipient, chainId]`; private witness is `(flavor, v, rho, idHash, predicate, pathElements[20], pathIndices[20])`. The circuit enforces five constraint groups:

- **(R)** `Num2Bits(128)` on `face` and on `v`.
- **(C)** `cm === Poseidon(5)(flavor, v, rho, idHash, predicate)`.
- **(M)** Tornado-style Merkle inclusion of `cm` under `noteRoot` at path `(pathElements, pathIndices)`: a chain of `Poseidon(2)` hashes with `Switcher` selectors ordering the left/right inputs at each level.
- **(N)** B-shape nullifier `nullifier === Poseidon(3)(rho, idHash, 4242)` – the constant tag 4242 domain-separates the B-path; the shipped `spend_a.circom V2` uses tag 4243 to keep A and B nullifiers structurally disjoint.
- **(B)** Ghost bind `face * 1 === v * 1` and a quadratic anchor `x*x` guarantees the prover committed to the value actually being redeemed (not an arbitrary smaller face).

`snarkjs` compiles the circuit to roughly 12,000 R1CS constraints; the 2^{15} pot is shared with mint. The on-chain `SpendGroth16Verifier` accepts a 5-element `pubSignals` tuple; the `SpendVerifierAdapter` presents the narrower `ISpendVerifier.verifySpend(proof, root, nullifier, face, recipient, chainId)` to the `Notes` contract.

7.3 Verifier Interfaces and Adapters

`Notes` is decoupled from the specific SNARK toolchain via two narrow interfaces:

```

interface IMintVerifier {
    function verifyMint(bytes proof, uint256 totalFace,
                      uint256[] commitments, address issuer) external view returns (bool);
}

interface ISpendVerifier {
    function verifySpend(bytes proof, uint256 root, uint256 nullifier,
                      uint256 face, address recipient, uint256 chainId)
        external view returns (bool);
}

```

Two implementations of each interface ship:

- **Stubs** (`StubMintVerifier`, `StubSpendVerifier`): governance-controlled booleans. Used in deployment bootstraps and unit tests that exercise state transitions unrelated to the SNARK itself.
- **Adapters** (`MintVerifierAdapter`, `SpendVerifierAdapter`): decode proof as `abi.encode(uint256[2], uint256[2][2], uint256[2])`, build the public-signals tuple (5 entries for mint – `[oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash]` – and 5 entries for spend), and forward to the snarkjs-generated `MintGroth16Verifier_N*` / `SpendGroth16Verifier`. The mint adapter is per-N: `mintVerifiers` is a mapping(`uint256 => IMintVerifier`)= keyed on `cms.length`, with one verifier registered per pinned batch size (recommended {16, 128, 1024}).

Swapping either verifier is a single governance call (`Notes.setMintVerifier(N, address)` / `Notes.setSpendVerifier(address)`); the `Notes` contract’s own storage and event surface are invariant.

7.4 Notes Contract

`src/Notes.sol` is constructed with (`buck`, `mintVerifierRegistry`, `spendVerifier`, `governance`). Its state is intentionally lean – the per-leaf Poseidon insertion math that Phase 7 had on chain now lives inside the mint SNARK, and the contract just records the SNARK-attested `newRoot`:

- **Verifier handles:** per-N `mintVerifiers[N]` (governance-swappable per pinned N), `ISpendVerifier spendVerifier` (governance-swappable singleton). No on-chain Poseidon precompile is required for mint – it remains relevant for legacy spend- side off-chain tree reconstruction tools, but the contract no longer calls it.
- **Audit and double-spend state:** mapping `nullifiers`, `uint256 noteFaceSum`.
- **Tree state:** `uint32 nextLeafIndex` and `uint256[ROOT_HISTORY_SIZE=30] roots` with `uint8 currentRootIndex` selecting the live root. `filledSubtrees`, `zeros[]`, and `commitmentExists` from Phase 7 are retired – the rolling tree state lives in the wallets’ local mirrors and inside the SNARK; duplicate commitments are economically self-punishing via the deterministic nullifier `Poseidon3(rho, idHash, 4242)` (one spend nullifies every copy of the same opening; Alice loses any BUCK she paid to mint duplicates).

`mint(proof, oldRoot, newRoot, nextLeafIndex, totalFace, cms)` asserts `cms.length > 0`, checks `oldRoot == roots[currentRootIndex]` and `nextLeafIndex == self.nextLeafIndex` (stale-state guards), recomputes `cmBatchHash = keccak256(abi.encodePacked(cms))`, dispatches to `mintVerifiers[cm]` (`oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash`), pulls `totalFace` BUCK via `transferFrom(caller, this, totalFace)`, writes `newRoot` into the ring buffer at `(currentRootIndex + 1) % ROOT_HISTORY_SIZE`, advances `nextLeafIndex` by `cms.length`, bumps `noteFaceSum` by `totalFace`, and emits a single `Minted(issuer, totalFace, startIndex, count, newRoot)` event. Per-leaf `Appended` events from Phase 7 are gone: the commitments themselves live in the transaction calldata for off-chain tree replayers and indexers to read.

`spend(proof, root, nullifier, face, recipient)` is unchanged from Phase 7: requires `recipient != 0`, `face > 0`, that `root` is still in the recent-roots window (`isAcceptedRoot(root)`), and that `nullifier` has not been seen. It forwards (`proof, root, nullifier, face, recipient, block.chainid`) to `spendVerifier.verifySpend`, marks the nullifier burned and decrements `noteFaceSum`.

before calling `IBuckCarrying(address(buck)).transferCarrying(recipient, face)` – the carrying path moves the pool’s average demurrage age to the recipient atomically with the face value. A `Spent(nullifier, face, recipient)` event closes the spend.

Phase 7-bis closes the same loop as Phase 7 but at *N-independent on-chain cost*: credit-backed BUCKs in via mint (380K + ~500 gas/leaf calldata) -> opaque commitments fold into the rolling root via the SNARK -> `=spend=` redeems any committed commitment against any recent root with a fresh nullifier, paying the recipient via `=transferCarrying=` so the SNARK’s `~face === v` invariant is preserved at the ERC-20 boundary. The pivot is documented in alberta-buck-notes-rollup-mint.org.

8 Integration: Full mint() and transfer() Walkthrough

Consider Alice, who owns a \$500K Calgary home (land: \$200K, structure: \$300K). Two insurers (LandCo, HomeCo) have issued BUCK_CREDITS:

Token	Insurer	Face Value	Depreciation	Activated	Current Value
#42	LandCo	200,000	NONE	200,000	200,000
#43	HomeCo	300,000	LINEAR, 200bp, 20% fl	150,000	141,000
Total					341,000

The structure has depreciated 6% over 3 years (LINEAR at 2%/yr applied to the 80% depreciable portion). Alice activated all of the land credit but only half of the structure credit.

8.1 Step 0: Identity Registration (One-Time)

Before Alice can interact with BUCK at all, she must hold a verified identity credential:

1. **KYC ceremony** (off-chain): a trusted issuer (the Alberta government, ATB Financial, etc.) verifies Alice’s documents and signs her identity scalar $m = H(identity_data)$ with their PS key, producing $\sigma = (\sigma_1, \sigma_2)$. Alice stores (m, σ) in her wallet.
2. **Derivation** (off-chain): Alice’s wallet rerandomizes σ into σ' , generates a fresh identity key pair (sk_{alice}, pk_{alice}) , encrypts her identity point $M = m \cdot G$ under pk_{alice} , and produces a NIZK proof π binding σ' to that ciphertext.
3. **On-chain registration**: Alice calls `IdentityRegistry.register(issuer, pk_alice, E_alice, sigma', pi)`. The contract verifies π and the PS pairing relation via the BN254 precompiles (235K gas, paid once for the lifetime of this Ethereum address). After this call, `~isVerified[alice] == true` and `issuerOf[alice] == issuer`.

Alice repeats steps 2-3 for every Ethereum account she controls; the issuer never sees those accounts.

8.2 Step 1: Mint

Alice calls `BUCK.mint(50000e18)`:

1. The contract first checks `identity.isVerified(alice) == true`.
2. `buckCredit.totalCurrentValue(alice)` returns 341,000e18.

3. `buckK.currentBuckK()` returns the current stabilization factor. **Phase 0:** this is just the static value the governance multisig last wrote. **Eventual PID build:** the controller fetches oracles – e.g. commodity basket at 1.02 USDC, BUCK at 0.99 USDC, error +0.03 -> `BUCK_K = 1.015`.
4. `maxLimit = 341,000 * BUCK_K`. Assume `BUCK_K = 1.015`, so `maxLimit = 346,115`.
5. Alice’s current balance is 280,000 BUCKs. After mint: `330,000 < 346,115`. OK.
6. Utilization after mint: `330,000 / 346,115 = 95.3%`.
 - Premium rate: $0.50\% + (0.953^2 * 4.50\%) = 4.59\%$.
 - Premium: `50,000 * 4.59% = 2,295` BUCKs to insurance pool.
7. Alice receives 47,705 BUCKs. Insurance pool receives 2,295 BUCKs.

8.3 Step 2: Approve Bob (Identity Handshake)

Alice wants to pay Bob 10,000 BUCKs through the same router pattern most ERC-20 flows use (`approve + transferFrom`). Bob is also registered.

1. Alice’s wallet reads Bob’s `pk_bob` from `IdentityRegistry`.
2. Off-chain, Alice’s wallet decrypts `E_alice` (it knows `sk_alice`), recovers M , and re-encrypts under pk_{bob} with fresh randomness r_b : $E_{bob} = (r_b \cdot G, r_b \cdot pk_{bob} + M)$.
3. The wallet produces a Chaum-Pedersen NIZK proof $\pi_{CP} = (e, s_1, s_2)$ that `E_bob` encrypts the same M registered in `E_alice`. The Fiat-Shamir challenge binds `msg.sender = alice`, `spender = bob`, and `block.chainid`.
4. Alice calls `BUCK.approve(bob, 10000e18, E_bob, pi_CP)`. The contract:
 - checks both parties are verified;
 - asks `IdentityRegistry.verifyApprove(alice, bob, E_bob, pi_CP)` – the registry reads `E_alice` from its own storage (so Alice can’t substitute a fake credential) and runs the CP check (~29K gas);
 - stores `_receiptFragments[alice][bob] = keccak256(E_bob)`;
 - records the standard ERC-20 allowance.
5. Off-chain, Alice’s Holochain hApp delivers the matching `identity_data` and E_{bob} ciphertext to Bob’s hApp. Bob’s wallet decrypts to recover M' and asserts $H(identity_data) \cdot G \stackrel{?}{=} M'$ – the two-channel binding from the Identity document.

8.4 Step 3: Bob Pulls the Transfer

Bob (or a contract Bob authorized) calls `BUCK.transferFrom(alice, bob, 10000e18)`:

1. `_spendAllowance(alice, bob, 10000e18)` debits the allowance Alice set above.
2. `_identityCheckedTransfer(alice, bob, 10000e18)` verifies both parties are still verified and that `_receiptFragments[alice][bob] != 0` (Alice did supply a CP-bound receipt at approve time).

3. Tokens transfer.
4. `BuckTransferReceipt(alice, bob, 10000e18, fromHash, toHash)` is emitted, with `fromHash` and `toHash` being keccak256 commitments over the corresponding ciphertexts. An auditor with subpoena power over either Alice or Bob can recover the underlying identities; an outside observer learns only that addresses `alice` and `bob` exchanged 10,000 BUCKs.

8.5 Step 4: Alice Denominates 125 BUCK as Notes

Separately, Alice wants to move some of her BUCKs off the public-ledger transfer path into private notes. Her wallet picks flavour `A2` and values `[100 BUCK, 25 BUCK]` (`totalFace = 125 BUCK`), draws fresh `rho[i]` and `idHash[i]` randomness, and – since the deployment pins $N \in \{16, 128, 1024\}$ – pads the live pair with 14 dummy openings whose `v_i = 0` so the batch lands on the smallest pinned circuit ($N^* = 16$). The wallet snapshots (`oldRoot`, `nextLeafIndex`) from chain, derives the depth-20 sibling paths for each of the 16 leaves *in submission order*, and runs `snarkjs groth16 fullProve` against `build/snark/mint_batch_N16/mint.wasm` and `mint_final.zkey`. The proof triple (`pA`, `pB`, `pC`) is abi-encoded as (`uint256[2]`, `uint256[2][2]`, `uint256[2]`) into `proofBytes`.

On-chain, Alice makes two calls:

1. `BUCK.approve(notes, 125e18, E_notes, pi_CP_notes)` – standard identity-aware approve. The `Notes` contract was bound at deployment under a Public Identity (`bindContract(notes, pk_notes, E_notes, true)` from `script/Deploy.s.sol`), so Alice’s wallet constructs `E_notes` by re-encrypting against the Notes pool’s published `m`. The CP proof itself is *still required* – the always-CP rule (see *Why approve Is Always CP-Bound*) applies regardless of spender flavour – and the on-chain receipt fragment Alice’s operator stores is Alice’s own `E_notes` re-encryption, openable on subpoena to show "Alice authorised the Notes pool to draw 125 BUCK."
2. `Notes.mint(proofBytes, oldRoot, newRoot, nextLeafIndex, 125e18, [cm[0], ..., cm[15]])` – the `MintVerifierAdapter` dispatches by `cms.length=16` to `mintVerifiers[16]` and forwards to the $N=16$ Groth16 verifier with public signals `[oldRoot, newRoot, nextLeafIndex, 125e18, keccak256(cms)]`. On success, 125 BUCK moves from Alice to Notes, `roots[]` advances to the SNARK-attested `newRoot`, `nextLeafIndex` advances by 16, and `noteFaceSum + 125e18 =`. Alice’s wallet keeps the live openings (`flavor`, `v`, `rho`, `idHash`, `predicate` for the two real notes) privately – those are the `NoteOpening` records she will need to spend. The 14 dummy openings are discarded.

8.6 Step 5: Alice (or her recipient) Spends a Note

Later, the holder of one of Alice’s notes – it could be Alice herself, or the bearer to whom she off-chain handed the opening – redeems `cm[0]` for 100 BUCK to address `recipient`:

1. The wallet reconstructs the Merkle path (`pathElements[20]`, `pathIndices[20]`) for `cm[0]` by replaying the `Minted` event log and pulling the calldata `cms[]` out of each batch mint transaction (or from a locally-cached tree).
2. `snarkjs groth16 fullProve` against `build/snark/spend/spend.wasm` and `spend_final.zkey` produces a proof binding `[noteRoot, nullifier, face, recipient, chainId] = [currentRoot, Poseidon(rho, idHash, 4242), 100e18, recipient, 1]` to the witness opening.

3. `Notes.spend(proofBytes, currentRoot, nullifier, 100e18, recipient)` – the contract checks the root is still in the recent-roots window, rejects a previously-seen nullifier, runs the adapter (~360K gas total), marks the nullifier burned, decrements `noteFaceSum`, and calls `BUCK.transferCarrying(recipient, 100e18)`. The recipient receives the full 100 BUCK and an amount-weighted share of the pool’s average demurrage age.

Later, HomeCo reappraises the structure at \$280K (updated `faceValue`). Alice’s credit limit drops. If her outstanding balance exceeds the new limit, the default insurance mechanism activates (or the Jubilee fund eventually releases the lien), and she can no longer mint BUCKs.

9 Gas Cost Analysis

Approximate gas costs (Ethereum mainnet, Solidity 0.8.x optimizer). The Notes rows are measured from the `MintVerifier.t.sol` and `SpendVerifier.t.sol` Foundry runs against the real Groth16 verifiers; the rest are architectural estimates for the eventual PID build.

Operation	Gas (approx)	Cost at 30 gwei	Notes
<code>BUCK_CREDIT.activate()</code>	~50K	~\$4	Single SSTORE + even
<code>BUCK_CREDIT.updateCredit()</code>	~80K	~\$6	Multiple SSTOREs (ins
<code>IdentityRegistry.register()</code>	~235K	~\$19	PS pairing + NIZK (on
<code>IdentityRegistry.verifyApprove</code>	~29K	~\$2	CP proof, per counterp
<code>BUCK.approve()</code> (identity)	~97K	~\$8	29K CP + 46K allowan
<code>BUCK.transfer</code> / <code>transferFrom</code>	~80K	~\$6	ERC-20 baseline + rece
<code>BUCK.transferCarrying()</code>	~60K	~\$5	Weighted-merge index,
<code>BUCK.mint()</code> (static <code>BUCK_K</code>)	~150K	~\$12	Enumerate credits + m
<code>BUCK.mint()</code> (with PID update)	~350K	~\$28	+ Oracle reads + PID c
<code>Notes.mint()</code> (mint_batch, N=1)	~461K	~\$37	Groth16 verify (N=1, ~
<code>Notes.mint()</code> (mint_batch, N=2)	~473K	~\$38	~237K gas/leaf – amort
<code>Notes.mint()</code> (mint_batch, N=4)	~500K	~\$40	~125K gas/leaf
<code>Notes.mint()</code> (mint_batch, N=8)	~562K	~\$45	~70K gas/leaf
<code>Notes.mint()</code> (mint_batch, N=16)	~624K	~\$50	~39K gas/leaf – shipped
<code>Notes.mint()</code> (mint_batch, N=32)	~1.12M	~\$90	~35K gas/leaf (verifier
<code>Notes.spend()</code> (Groth16, B-shape)	~360K	~\$29	Groth16 verify + trans
<code>Notes.spendACP()</code> (Groth16 + CP-DLEQ, A-shape)	~410K	~\$33	Groth16 verify + verify (~36K via EIP-196) +
<code>BUCK_CREDIT.currentValue()</code>	0 (view)	Free	Pure computation, no s
<code>BUCK_K.currentBuckK()</code>	0 (view)	Free	Read cached value

The PID update cost (~200K gas additional) is amortized across all mints within a `dT` window. With a 1-hour `dT`, only the first mint after each hour pays for the oracle reads and PID computation.

For lower gas costs, the same contracts can be deployed on Ethereum L2s (Arbitrum, Optimism, Base) where gas is 10-100x cheaper, or on alternative EVM chains. The contract architecture is chain-agnostic.

10 Security Considerations

10.1 Oracle Manipulation

- **TWAP resistance:** The BUCK/USDC price uses a Uniswap V3 TWAP (30+ minutes), requiring sustained capital commitment to manipulate.
- **Chainlink feeds:** Commodity prices from Chainlink’s decentralized oracle network, with built-in deviation thresholds and heartbeat checks.
- **Staleness checks:** `_getBasketCost()` should verify `updatedAt` is within an acceptable window (e.g., 1 hour) and revert if stale.

10.2 Credit Enumeration DoS

A malicious user could create many tiny BUCK_CREDITS to make `totalCurrentValue()` gas-prohibitive. Mitigations:

- Minimum face value for BUCK_CREDIT creation.
- Cap on credits per account (e.g., 20).
- Off-chain aggregation with Merkle proof (advanced).

10.3 Insurer Trust

The insurer can reduce `faceValue` at any time, potentially pushing accounts into default. This is by design – it reflects real-world reappraisal risk. Mitigations:

- Insurance pool covers default events.
- Multiple competing insurers provide market discipline.
- Governance can blacklist abusive insurers.
- Time-locks on insurer updates (e.g., 7-day delay for reductions > 20%).

10.4 Reentrancy

Standard OpenZeppelin `ReentrancyGuard` on `mint()` and `activate()`. All state changes occur before external calls.

11 Future Approaches

This ERC-721 implementation prioritizes clarity and correctness. Subsequent documents will explore:

11.1 ERC-1155: Semi-Fungible Batch Credits

If standardized credit classes emerge (e.g., "Calgary residential, HomeCo, LINEAR 200bp"), multiple properties in the same class could share a token ID with individual amounts. Benefits:

- Batch operations (`safeBatchTransferFrom`) for portfolio management.
- Reduced gas for accounts with many same-class credits.
- Natural fit for insurance tranches within a single credit.

Challenges: per-token depreciation start dates require additional state mapping, partially negating the fungibility benefit.

11.2 ERC-3643 (T-REX): Regulated Security Token

For production deployment under regulatory oversight:

- **ONCHAINID**: Native identity registry for KYC/AML compliance.
- **Compliance modules**: Programmable transfer restrictions (e.g., credits can only transfer to verified Alberta residents).
- **Agent roles**: Insurer as agent with granular permissions (update, freeze, force-transfer).
- **Recovery**: Lost-key recovery through identity verification.

This is the likely production standard if Alberta implements BUCK under provincial regulation.

11.3 ERC-4626: Vault-Based Insurance Pool

While a poor fit for BUCK_CREDIT itself, ERC-4626 is well-suited for the `InsurancePool`:

- Premium deposits as `deposit()` / mutual insurance shares as `shares`.
- `convertToAssets()` / `convertToShares()` for transparent share pricing.
- Standard DeFi composability (yield aggregators, lending protocols).

11.4 L2 and Cross-Chain Deployment

The contracts are EVM-compatible and can deploy on Arbitrum, Optimism, Base, or Polygon with minimal changes. Cross-chain BUCK transfers via canonical bridges or LayerZero/Axelar would enable multi-chain liquidity while maintaining a single BUCK_K oracle on the settlement layer.

12 Implementation Plan

This document specifies the **eventual** shape of the system. The first deployment ships in narrow phases so the cryptographic identity layer – the single largest piece of new ground in the design – can land first, against minimal stubs of the surrounding contracts. Each phase ends in a self-contained, deployable state with passing tests. The status table below reflects the repo at the header date; the phase subsections document the intent of each phase as originally scoped and are annotated with what the current code actually ships.

The phase set has grown since the original v0.5 plan in two directions. *Backward*, Phase 7-bis landed the in-circuit Merkle-insertion batch-mint pivot documented in alberta-buck-notes-rollup-mint.org – now shipped as `mint_batch.circom` with per-N pinned verifiers in {1, 2, 4, 8, 16, 32}. Phase 8 (A-flavor spend with key-pair binding) also shipped, in two layers: `spend_a.circom` V2 (in-circuit Poseidon-8 idHash binding gate) plus off-chain Chaum-Pedersen DLEQ verified in `IdentityRegistry.verifySpendCP` via EIP-196 BN254 precompiles, all glued by the new `Notes.spendACP` entry point. *Forward*, the path to live BUCK_K (Phase 10/11) and dynamic BUCK_CREDIT-backed minting (Phase 12) is sequenced behind one remaining Notes deliverable: serialized B1 sub-notes (Phase 9) per alberta-buck-notes-serialized.org. Both companion memos have been reviewed for cryptographic correctness; see *Cryptographic Correctness Review of Companion Memos* below.

12.1 Phase Completion Status (current)

Phase	Scope	Status
Phase 0	Stub <code>BuckCredit</code> + <code>BuckKControllerStatic</code>	done – <code>BuckKControllerStatic</code>
Phase 1	BN254.sol helper library	done – <code>ecAdd/ecMul/ecPairing</code>
Phase 2	<code>IdentityRegistry.sol</code>	done – PS + NIZK + CP all on
Phase 2-A	Simulated issuer (<code>alberta_buck.wallet.issuer</code>) + tests	done – <code>Issuer</code> class with PS iss
Phase 3	Identity-aware <code>Buck.sol</code>	done – CP-bound approve, bilat
Phase 4	Public-Identity contracts	done – <code>bindContract(target,</code>
Phase 5	Notes scaffolding + stub verifiers	done – <code>Notes.sol</code> + <code>StubMintV</code>
Phase 6	Groth16 mint circuit (<code>mint.circom</code> , N=2) + adapter	done – <code>MintGroth16Verifier, M</code>
Phase 7	B-flavor spend circuit + nullifier reveal	done – <code>spend.circom</code> , <code>SpendGro</code>
Phase 7-bis	Batch-mint pivot: in-circuit Merkle insertion	done – <code>mint_batch.circom</code> , per
Phase 8	A-flavor spend (<code>spend_a.circom</code> V2 + off-chain CP-DLEQ)	done – <code>spend_a.circom</code> V2 (in-
Phase 9	Serialized B1 sub-notes (<code>spend_serialized.circom</code> + bitmap)	pending – POC validated to N ₁
Phase 10	Live <code>BuckKController</code> (PID + AMM-pool oracles)	pending – contract skeleton in-t
Phase 11	BUCK liquidity bootstrap + BUCK/USDC AMM pool	pending – closes the loop on Ph
Phase 12	BUCK_CREDIT insurance integration in <code>Buck.mint</code>	pending – per-tx credit-limit de
Phase 13	Production MPC trusted-setup ceremony	pending – replaces dev entropy

Current forge suite: 202 unit tests pass (across 13 test suites); one `BuckKControllerForkTest` requires a live mainnet RPC and is skipped offline.

12.2 Cryptographic Correctness Review of Companion Memos

Two design memos drive the immediate path forward. Phase 7-bis (batch-mint) and Phase 8 V2 (A-spend) are both shipped; only Phase 9 (serialized B1 sub-notes) remains as an outstanding Notes deliverable. Both memos were reviewed for cryptographic correctness on 2026-04-24; both verdicts are *sound*, with one explicit operational trust assumption (`K_fam` destruction in serialized B1) that is *symmetric* to a pre-existing one (issuer-retained `rho` in single-note B1) and is documented in the Phase 9 wallet-UX requirement.

12.2.1 alberta-buck-notes-rollup-mint.org – Phase 7-bis batch-mint design

Verdict: cryptographically sound; integrated into shipped code. The in-circuit Poseidon Merkle insertion replaces the per-leaf `Notes._insert` ladder with one SNARK-attested `newRoot` per batch.

- **Public-input layout** (`oldRoot`, `newRoot`, `nextLeafIndex`, `totalFace`, `cm[0..N-1]`) is binding-complete: the contract's `oldRoot` = `roots[currentRootIndex]` = guard plus the SNARK's derivation of `newRoot` from `oldRoot` and the folded leaves chains the on-chain root state to the SNARK-attested transition without a trusted off-chain step.
- **Dual Merkle walk** per leaf – `priorWalk` consumes `ZERO_VALUE` at the insertion slot to attest `oldRoot`; `postWalk` consumes `cm[i]` to advance `rollingRoot` – is the correct in-circuit dual of the shipped on-chain insert. Both walks share the same siblings, supplied by the prover's local `filledSubtrees` mirror.
- `nextLeafIndex` as a public input forces wallet/chain agreement on tree size and prevents the prover from misplacing leaves at a stale index.
- **Hostile-Alice analysis** A1 through A12 holds. A3 (duplicate `cm` across mints) correctly migrates from explicit revert to economic self-enforcement: the deterministic `Poseidon-3(rho, idHash, 4242)` nullifier means duplicate openings are spendable exactly once; the issuer pays `totalFace` for each duplicate and forfeits all but one. A6 (lying about `newRoot`) is closed by the SNARK constraint set; A7 (front-running) reverts on the `oldRoot` guard with no value lost.
- **Affine cost model** `mint_tx_gas(N)` $\sim 125K + 31K * N$ = empirically fits the measured `N=16` (624K) and `N=32` (1.12M) Foundry runs; per-note cost asymptotes at $\sim 31K$.
- **Pinned-N policy** {16, 32} on stock L1 (both fit the EIP-170 24 KB ceiling at 14.5 KB and 20.8 KB respectively), promote to {16, 32, 128} via calldata-IC delivery, recursion above `N=512` is the natural sequencing. Currently shipped: per-N pinned verifiers at {1, 2, 4, 8, 16, 32}; the smaller sizes are useful for unit tests and small-batch wallet flows even after the L1 ceiling is reached.

12.2.2 alberta-buck-notes-serialized.org – Serialized B1 sub-notes design

Verdict: cryptographically sound, with `K_fam` destruction as the single operational trust assumption (symmetric to existing single-note B1 issuer-race over `rho`). Mint-circuit reuse is exact; only the spend circuit and the double-spend storage diverge.

- **Construction:** `subRoot = MerkleRoot(s_0..s_{N-1})`, `predicate_fam = Poseidon-2(predicate_base, subRoot)`, `cm_parent = Poseidon-5(flavor, v, rho, idHash, predicate_fam)`. Because `cm_parent` is an ordinary `Poseidon-5` output, `mint_batch.circom` hashes it identically to a single-note B1 leaf – *no new mint circuit, no new trusted setup, no new pinned-N variants*.
- **Witness-and-verify discipline:** the spend circuit does NOT invert `s_i -> i` (which would require breaking the PRF *and* would let an attacker enumerate every sibling). The recipient witnesses (`i`, `s_i`, `path_i_in_subRoot`) privately; the circuit verifies `MerkleOpen(subRoot, i, s_i, path) === true`. `i` is emitted as a public signal and used by the contract to flip `bitmap[cm_parent]` bit `i`.
- **K_fam destruction** is the documented operational trust assumption. An issuer retaining `K_fam` can produce (`i`, `s_i`, `path`) on demand for any sub-note and impersonate recipients. This is *symmetric* to the existing single-note B1 issuer-race – an issuer who retains `rho` can race the recipient at spend time – and is enforceable only by wallet UX / HSM attestation, not by the cryptography itself. Promoted to a Phase 9 deliverable.

- **B1-only restriction:** each spend reveals `cm_parent`, linking co-spent siblings. For bearer notes the linkage is free (denom + family membership are not secrets in a bearer model); for A1/A2 the linkage would defeat the privacy property. Correctly scoped.
- **Per-parent bitmap nullifier** amortizes to $\sim 2.96\text{K}$ gas per sub-note (SLOAD + warm SSTORE) vs 20K cold for the single-note path – $\sim 7\text{x}$ storage reduction at the asymptote. The *bit-flip squat* attack is closed by the fact that only `Notes.spendSerialized` writes bits, and that path requires a Groth16 proof binding (`cm_parent`, `i`) to the recipient.
- **Brute-force resistance** is 2^{128} (Poseidon preimage / second-preimage), identical to the single-note path. Serialization adds zero attack surface at the "forge a note I don't own" level.

The `scripts/snark/serialized_notes_poc.py` driver validates the construction end-to-end (issuer-mint, recipient-verify, spend-predicate, forgery tests) at `N_fam` in $\{256, 1024, 4096\}$. Porting the predicate to `circuits/spend_serialized.circom` is the remaining implementation work.

The two memos are mutually orthogonal: rollup-mint amortizes *publishing* N commitments into `noteRoot`; serialized amortizes *producing* N bearer notes from *one* commitment. Together they drop marginal per-note mint cost by 2-3 orders of magnitude vs. the shipped per-leaf design.

12.3 Testing Strategy: Three Layers

Tests are stratified by what they actually need from the EVM, so the cryptographic and accounting work can iterate without ever touching a network:

1. **Local-EVM unit tests (default).** Foundry's in-memory EVM ships the BN254 precompiles (`ecAdd 0x06`, `ecMul 0x07`, `ecPairing 0x08`) and all the EIP-2929 / Cancun behavior we need. Every contract in this document – `BUCK_CREDIT`, `BUCK`, `IdentityRegistry`, the BN254 helper, the static `BUCK_K` – is fully exercisable without RPC, on a clean state, in milliseconds per test.
2. **Pinned mainnet/Sepolia forks (only the eventual PID `BUCK_K`).** When the production `BuckKController` ships and needs to read Chainlink price feeds and the Uniswap V3 `BUCK/USDC TWAP`, run the relevant tests with `--fork-url ${RPC} --fork-block-number ${N}`. Pinning the block number is the determinism knob: the EVM state at block N is immutable, and Foundry caches the RPC responses under `~/.foundry/cache/rpc/` – after the first run the suite is offline and bit-reproducible. `vm.mockCall` covers pathological oracle paths (stale feeds, manipulated TWAPs) without needing to find a block at which they actually occurred.
3. **Differential parity with the Python reference.** The cryptographic surface is described by an executable Python reference (see **Off-Chain Wallet** below). CI regenerates the canonical JSON test vectors from that reference and `git diff --exit-code`'s them against committed copies; the Solidity tests load the same vectors via `vm.readFile` and assert equality. Any drift between the spec, the wallet, or the Solidity tells us *immediately* which of the three is wrong.

The deliberate exclusion: no `vm.ffi` shelling out to Python at test time. That trades determinism, speed, and CI portability for a coupling we don't need – the wallet emits vectors once, the Solidity reads them many times.

12.4 Off-Chain Wallet (`alberta_buck.wallet`, Python)

The off-chain components – KYC ceremony, credential derivation, Chaum-Pedersen proving, and (later) SNARK witness generation for Notes – are implemented as a Python package, `alberta_buck.wallet`, with a CLI front-end. Python is the natural choice: the existing references in Identity Example and Proofs are already Python, `py_ecc` ships a complete BN254 (alt_bn128) implementation matching Ethereum’s precompiles, and the package is invoked the same way every other dev tool in this repo is (`nix develop --command python3 -m alberta_buck.wallet ...`).

The wallet has four jobs:

- **Reference implementation** of the cryptography in Identity, the place where the spec is ground-truthed.
- **Test-vector emission:** a CLI verb (`wallet emit-vectors`) writes a canonical JSON file under `test/vectors/` containing the inputs and expected outputs of every on-chain proof check. Foundry tests load this file and assert agreement with the Solidity verifier.
- **End-to-end developer surface:** a CLI verb (`wallet sign`, `wallet register`, `wallet approve`) produces the calldata an end user’s wallet would submit, so integration tests (and humans) can drive the contracts the same way a real wallet would.
- **Simulated counterparties:** `alberta_buck.wallet.issuer` models the off-chain government / institutional issuer (PS keypair, identity-record canonicalization, signed credential delivery, optional ElGamal-encrypted transmission to the applicant, in-memory revocation log). This is the entity the wallet’s `register` verb effectively *receives a credential from* before rerandomizing and posting to `IdentityRegistry`. See Phase 2-A for the implementation status.

The wallet does **not** hold private keys for production users; it is a deterministic reference and a test driver. A production wallet would reuse the same primitives behind a different storage and UI.

12.5 Phase 0: Stub the Surrounding Contracts

Goal: Buck can be developed and tested against a minimal, deterministic surface for `BuckCredit` and `BuckK`. All real moving parts (depreciation curves, PID, oracles) are deferred.

- `BuckKController.sol` -> replace with `BuckKControllerStatic.sol`:
 - State: `uint256 public buckK` (18 decimals, default `1e18`).
 - Functions: `currentBuckK()` `view` returns `(uint256)`, `compute()` (no-op compat shim), `setBuckK(uint256)`, `transferGovernance(address)` (both governance-only).
 - Drop all PID state, Chainlink basket, and Uniswap TWAP code. The full PID implementation documented above remains the design target – it slots back in once the identity layer is stable. The design-target `BuckKController.sol` also ships (unit-tested with mocked feeds); the live-oracle fork test is gated on RPC availability.
- `BuckCredit.sol` -> keep the existing implementation, but treat only one function as a load-bearing interface for Phase 0+: `totalCurrentValue(address holder)` `external view` returns `(uint256)`. No depreciation rework, no insurer-flow churn.

Deliverables: passing Foundry tests that mint, set new `BUCK_K` values, and watch credit limits re-derive correctly.

12.6 Phase 1: BN254 Helper Library

Goal: a single Solidity library that wraps the three BN254 precompiles and exposes the typed operations the identity contracts need.

- `src/crypto/BN254.sol` with:
 - `G1Point`, `G2Point` structs and constants for `g1`, `g2`.
 - `add(G1Point, G1Point) -> G1Point` via `ecAdd` precompile (0x06).
 - `mul(G1Point, uint256) -> G1Point` via `ecMul` precompile (0x07).
 - `pairingProd3(...)` and `pairingProd4(...)` helpers via `ecPairing` precompile (0x08).
 - Pure-Solidity Fiat-Shamir helpers: `fsRegistration(...)` and `fsChaumPedersen(...)`, each wrapping a `keccak256` over the canonical encoding from the Identity document.
 - High-level checks: `checkPSSignature`, `checkRegistrationNIZK`, `checkChaumPedersen`.

Deliverables: differential tests against the Python reference in `alberta-buck-identity-example.org` (same curve, same encoding, same Fiat-Shamir transcript). `nix develop --command python3` drives the reference; Foundry `vm.ffi` or pre-baked vector files feed the Solidity tests.

12.7 Phase 2: IdentityRegistry Contract

Goal: ship the on-chain trust anchor exactly as specified in the IdentityRegistry section.

- `src/IdentityRegistry.sol` implementing `register`, `bindContract`, `verifyApprove`, `trustIssuer`.
- All proofs verified using BN254 helpers from Phase 1.
- Tests:
 - End-to-end registration with the Python reference vectors (positive case + the four counter-examples from `alberta-buck-identity-example.org`).
 - `register` rejects untrusted issuers.
 - `register` rejects double-registration of the same address.
 - `verifyApprove` accepts a valid Chaum-Pedersen proof and rejects every documented tampered case (wrong message, wrong randomness, wrong public key, replayed challenge).
 - Gas measurements within ~15% of the published estimates (235K `register` / 29K `verifyApprove`).

Deliverables: deployable registry, passing test suite, documented gas measurements.

12.8 Phase 2-A: Simulated Issuer (Government Registry) + Test-Credential Pipeline

Goal: model the off-chain side of credential issuance – the entity an applicant submits identity documents to, which signs them with PS and (optionally) returns the credential ElGamal-encrypted to the applicant’s public key. Without this layer the existing tests have no way to drive `IdentityRegistry` other than from inline `ps_keygen` + `ps_sign` calls in test fixtures, which collapses the issuer’s role into the wallet’s role and makes multi-issuer / rotation / revocation scenarios awkward.

The on-chain trust anchor is unchanged: `IdentityRegistry.trustIssuer(addr, pk)` keys the issuer's PS public key by Ethereum address and `revokeIssuer(addr)` clears it. This phase adds the matching off-chain entity – same PS keypair, same address, plus the issuance ceremony itself.

- `alberta_buck/wallet/issuer.py`:
 - `Issuer` dataclass: `issuer_id` (string label, e.g. "atb-financial-ca"), `issuer_addr` (Ethereum address used as the on-chain trust anchor), `keypair` (PS), simulated revocation set, in-memory issuance log.
 - `Issuer.setup(issuer_id, issuer_addr, rng)` class-method for deterministic test setup.
 - `issue(identity_fields, applicant_addr, applicant_pk=None, rng) -> IssuedCredential`: overwrites the `issuer_id` field in the submitted record (so applicants cannot smuggle a different issuer label into the canonical), canonicalizes, computes m , signs σ , and
 - when `applicant_pk` is given – ElGamal-encrypts $M = m \cdot G$ under it for confidential delivery. The wallet decrypts on receipt to confirm M before rerandomizing σ .
 - `revoke(addr)` / `reset(addr)` / `issuance_log()` / `verify_credential(cred)` helpers.
 - `rerandomize_for_registration(cred, rng)` – convenience wrapper for the wallet-side step that produces σ' before calling `IdentityRegistry.register`.
- `alberta_buck/test/test_issuer.py`: 15 tests covering keygen determinism, single-credential round-trip, `issuer_id` overwrite invariant, ElGamal delivery decryption, revocation, multi-issuer non-cross-verification, and the full end-to-end pipeline (issue -> rerandomize -> ElGamal encrypt -> `registration_prove` -> `registration_verify`).
- Future: replace the inline `ps_keygen` calls in `alberta_buck/wallet/vectors.py` and the Foundry-side identity test vectors with `Issuer.setup(...).issue(...)` so the canonical JSON emitted to `test/vectors/` reflects an actual issuance ceremony rather than a synthetic one. This is mechanical and can be done incrementally – vectors stay byte-identical when seeded the same way.

Cryptographic note: ElGamal *re-encryption* proper (Alice -> Bob during `approve`) is wallet-side and remains in `alberta_buck.wallet.chaum_pedersen`. The issuer's optional ElGamal step is a one-shot encryption to the applicant; no Chaum-Pedersen proof is required because the issuer is trusted to encrypt to the address the applicant nominated, and the applicant verifies M decrypts to $m \cdot G$ for the m they intended to receive.

Deliverables: `issuer.py` module exporting `Issuer` and `IssuedCredential`; `test_issuer.py` green; the wallet's `__init__.py` re-exports the new symbols so downstream tests can write `from alberta_buck.wallet import Issuer`.

12.9 Phase 3: Identity-Aware Buck ERC-20

Goal: replace the current `Buck.sol` with the identity-aware version specified in the BUCK section.

- Add `IIdentityRegistry` interface and constructor wiring.
- mint gates on `isVerified`.
- `approve(spender, amount, E_bob, pi_CP)` with the parameterless `approve(address, uint256)` reverting.

- `transfer` / `transferFrom` via `_identityCheckedTransfer` with bilateral checks and the `BuckTransferReceipt` event.
- `_receiptFragments` storage and `_identityHash` helper (keccak of bound (pk, E)).

Tests:

- Two registered users can complete the full `approve` -> `transferFrom` cycle and observe a `BuckTransferReceipt` whose hashes match the off-chain ciphertexts.
- Unregistered mint reverts.
- `transfer` to an unregistered recipient reverts.
- `transfer` without a prior CP-bound `approve` reverts when neither party is bound under a Public Identity.
- Transfers to/from a Public-Identity contract may fall back to the bound `_identityHash` when no per-pair receipt fragment exists; EOA<->EOA encrypted transfers MUST have a prior CP receipt.

12.10 Phase 4: Public-Identity Contract Support

Goal: AMM pools, vaults, and treasuries can interact with BUCK without needing to generate Chaum-Pedersen proofs.

- Confirm `bindContract(target, pk, E, isPublicIdentity_=true)` is invocable only against deployed contracts (`target.code.length > 0`) and is first-binder-wins; atomic `deploy+bind` is available via `BuckAwareDeployer.deployAndBind` / `deployAndBindCreate2`.
- Add a small `MockAMMPool` test contract whose `deployer bindContract=s` it as a `Public-Identity` contract with the operator's published `=(pk, E)`, then participates in `approve/=transferFrom` flows on both sides (taker and maker).
- Verify the receipt event uses the bound `_identityHash` as the public side's fragment when no per-pair CP fragment exists, and the CP-bound ciphertext on the counterparty side.

12.11 Phase 5: Notes Scaffolding

Goal: lay the structural groundwork for BUCK Notes – a commitment pool, a pluggable mint verifier interface, and the off-chain wallet helpers – behind a stub verifier so the end-to-end flow can be exercised without yet shipping the SNARK.

Shipped:

- `src/Notes.sol` – the contract documented in the Notes section above. The initial Phase-5 deployment used a flat `commitments[]` append list behind a stub verifier; the incremental Poseidon Merkle tree and 30-slot recent-roots ring buffer landed alongside Phase 7.
- `src/IMintVerifier.sol` / `src/ISpendVerifier.sol` – the narrow verifier interfaces. Decoupling `Notes` from the SNARK toolchain was the key Phase-5 invariant so the Phase-6 / Phase-7 verifiers land as constructor-argument swaps.

- `src/StubMintVerifier.sol` / `src/StubSpendVerifier.sol` – governance-toggable boolean verifiers used by all Notes state-machine tests.
- `alberta_buck.wallet.notes` (Python) – `NoteOpening` dataclass, flavour constants, keccak-based placeholder commitment / nullifier helpers with domain-separating tags, `id_payload_a1/a2/b1` helpers matching the expected circuit public-input layout. 13 pytest tests.
- `test/Notes.t.sol` – 16 tests including happy path, duplicate detection, zero-commitment rejection, empty batch, verifier disabled, external rejecting verifier, missing approval, missing balance, state-atomicity on transfer failure.

The Python wallet’s `note_commitment` is a keccak-based placeholder; the on-chain contract stores opaque `uint256` commitments without re-hashing, so swapping keccak for Poseidon is an off-chain wallet change only.

12.12 Phase 6: Groth16 Mint Circuit + Verifier

Goal: turn the stub into a cryptographically load-bearing verifier. The mint contract must refuse to append a batch unless a real Groth16 proof ties `totalFace` to the sum of the (flavour, value, rho, `idHash`, predicate) openings, and each `cm[i]` to `Poseidon(5)` of its opening.

Shipped:

- `flake.nix` adds `circom` to the dev shell and bootstraps npm dev dependencies on first entry.
- `package.json` pins `circomlib` 2.0.5, `circomlibjs` ^0.1.7, `snarkjs` 0.7.5, `ethers` ^6.13.4.
- `circuits/mint.circom` – $N = 2$ mint circuit (see Notes section for the signal table), with `Num2Bits(128)` range bounds on `totalFace` and each `v[i]`. ~3,500 R1CS constraints.
- `scripts/snark/setup.sh` – `circom` compile, 2^{15} powers-of-tau (shared between mint and spend), `groth16` phase-2 setup, export verification key + Solidity verifier (`MintGroth16Verifier`, `SpendGroth16Verifier`). Dev-only entropy.
- `scripts/snark/prove_mint.js` – witness builder (two A2 notes, `v[100,25]e18=`), seeded LCG randomness, `snarkjs.groth16.fullProve`, pB Fp2-coordinate swap, abi-encoded `proofBytes`, and sanity-check `snarkjs.groth16.verify`.
- `src/MintGroth16Verifier.sol` – `snarkjs`-generated verifier.
- `src/MintVerifierAdapter.sol` – `MINT_BATCH_SIZE=2` adapter implementing `IMintVerifier`.
- `test/MintVerifier.t.sol` – 7 tests: happy path (real proof accepted), three tamper attacks (`totalFace`, `cm[0]`, batch size) all reverting with `Notes: bad mint proof`, malformed proof-bytes revert at decode, adapter constructor rejects zero.

The circuit is pinned at $N=2$. A variable- N deployment re-templates both the circuit and the adapter’s public-signals arity.

12.13 Phase 7: Spend Circuit + Nullifier Reveal

Goal: cryptographically redeem a minted commitment against an on-chain Merkle root under a fresh nullifier, paying out an equal face value of BUCK via `transferCarrying` so the SNARK's `face == v` invariant is preserved at the ERC-20 boundary.

Shipped:

- `circuits/spend.circom` – single-note B-shape redemption (~12,000 R1CS constraints). Public inputs [`noteRoot`, `nullifier`, `face`, `recipient`, `chainId`]; five constraint groups (range, commitment, Merkle inclusion at depth 20, nullifier determinism `Poseidon(3)(rho, idHash, 4242)`, value conservation).
- `circuits/spend_a.circom V2` – A-shape redemption circuit (~13,300 R1CS constraints; 6152 non-linear + 7140 linear). Public inputs [`noteRoot`, `nullifier`, `face`, `recipient`, `chainId`, `E_n.R.x`, `E_n.R.y`, `E_n.C.x`, `E_n.C.y`] (9 total); adds the A-tag nullifier `Poseidon(3)(rho, idHash, 4243)`, the flavor-in-{1,2} gate, and the in-circuit `Poseidon-8` binding constraint `Poseidon-8(E_n.R.x, E_n.R.y, E_n.C.x, E_n.C.y, issuerData[0..3]) == idHash` (~265 R1CS) that pins the publicly-revealed `E_n` to the leaf at mint time.
- `src/SpendGroth16Verifier.sol` and `src/SpendVerifierAdapter.sol` – snarkjs-generated verifier and the narrow `ISpendVerifier` adapter for B-shape spends.
- `src/SpendAGroth16Verifier.sol` and `src/SpendAVerifierAdapter.sol` – snarkjs-generated verifier and the narrow `ISpendAVerifier` adapter for A-shape (V2) spends; the adapter packs the 9 public signals in order and calls the underlying `verifyProof`.
- `src/Notes.sol` extended with the Tornado-style incremental Poseidon Merkle tree (depth 20, `TREE_DEPTH` precomputed empty-subtree roots, 30-slot `roots[]` ring buffer), and three spend entry points: `spend(proof, root, nullifier, face, recipient)` for B-shape, `spendACP(proof, root, nullifier, face, recipient, E_n, cpProof)` for A-shape (which calls both `spendAVerifier.verify` and `identityRegistry.verifySpendCP` atomically), and a governance setter `setIdentityRegistry` guarded by the same `onlyGovernance` modifier as the verifier setters.
- `src/PoseidonT3Bytecode.sol` – a helper that deploys the 2-input Poseidon precompile from raw bytecode generated by `circomlibjs`; `script/Deploy.s.sol` deploys it and passes the address to `Notes`'s constructor.
- `src/Buck.sol` extended with continuous demurrage (`cumIndex`, `BASE_RATE_PER_SEC`, per-account `_indexAtLastTouch`) and `transferCarrying(to, amount)` – a single entry point that performs a weighted-index merge on the recipient without burning a fee, so a note redemption pays the recipient `face`, not `face - fee`. (The per-account `FeeTreatment` enum that earlier drafts proposed has been replaced by this simpler single-function approach; the `Notes` pool does not need a flag because `transferCarrying` is the only path it calls.)
- `test/SpendVerifier.t.sol` and `test/BuckDemurrage.t.sol` cover the B-spend path: happy path, double-spend, unknown root, tampered recipient/face/nullifier, `chainId` mismatch, zero-recipient, and the weighted-merge invariant that the recipient's age equals the pool's average after spend.
- `test/SpendAVerifier.t.sol` covers the A-spend path (V2; 26 tests): happy path, double spend, A-tag-disjoint-from-B-tag, every SNARK-public-input tamper (including all four `E_n`

coordinates), every CP-DLEQ tamper (e, s, T1, T2), unregistered/wrong-spender rejection via `verifySpendCP`, and governance lockdown for both `setSpendAVerifier` and `setIdentityRegistry`.

Pending (now sequenced as their own phases below):

- **Phase 7-bis:** in-circuit Merkle-insertion batch-mint pivot. Now *done* – see *Phase 7-bis: Batch-Mint Pivot* below.
- **Phase 8:** A-flavor spend (`spend_a.circom` V2 with in-circuit Poseidon-8 binding + off-chain CP-DLEQ via `Notes.spendACP`). Now *done* – see *Phase 8: A-flavor Spend* below.
- **Phase 9:** serialized B1 sub-notes (`spend_serialized.circom` + per-parent bitmap).
- **Phase 10:** live `BuckKController` deployment with AMM-pool oracles (XAU/USDC, BUCK/USDC, etc.).
- **Phase 11:** BUCK liquidity bootstrap + BUCK/USDC AMM pool deployment.
- **Phase 12:** BUCK_CREDIT insurance integration in `Buck.mint` (dynamic credit-limit deduction + simultaneous insurance purchase).
- **Phase 13:** production MPC trusted-setup ceremony.

12.13.1 Resolved Items from the Notes Correctness Review

The Phase 6 → Phase 7 transition surfaced four correctness obligations originally listed as open questions in `alberta-buck-proofs.org` (Part IV). All four are now resolved in the shipped contracts:

1. **Num2Bits range checks on `v[i]` and `totalFace` in `circuits/mint.circom`** – *resolved*. `vRange[i] = Num2Bits(128)` and `totalRange = Num2Bits(128)` are wired in `mint.circom`; `scripts/snark/setup.sh` and `MintGroth16Verifier` have been refreshed. The spend circuit re-derives the matching range bound on its own `v` (`vRange = Num2Bits(128)`) and on the public face (`faceRange = Num2Bits(128)`), closing the field-overflow corridor end-to-end.
2. **Explicit batch identifier in `Notes.mint`** – *still an open design question, not a correctness bug*. The shipped `Minted(issuer, totalFace, startIndex, count, newRoot)` event carries a root per batch, which an indexer can Poseidon-commit to if an auditable `batchId` is wanted. Revisit if regulatory analytics need an on-chain `batchIds` map.
3. **Phase-7 spend circuit and on-chain support** – *resolved*. Shipped as described above: the Tornado-style incremental Poseidon tree lives in `Notes.sol`, nullifiers are computed in-circuit as `Poseidon(3)(rho, idHash, 4242)`, and value conservation is enforced via the 128-bit range bound plus the commitment reconstruction constraint.
4. **Per-account fee-treatment mode** – *superseded*. The earlier draft added an enum `FeeTreatment {Deducting, Carrying}`, a per-account `_feeMode` mapping, a `BuckDerived()` bytecode marker, a `balanceOf(account) == 0` precondition for mode changes, and a `FeeCarried` event. The shipped solution is simpler: a single `transferCarrying(to, amount)` method on `Buck.sol` that performs the weighted-merge index update atomically, callable by any contract (today only `Notes` calls it; future escrow/payroll contracts may too). No per-account mode, no bytecode marker, and no balance precondition are needed because the weighted merge is continuous – the recipient simply absorbs a proportional share of the sender’s unsettled age.

12.14 Phase 7-bis: Batch-Mint Pivot (in-circuit Merkle insertion)

Goal: collapse the per-leaf on-chain Poseidon ladder of Phase 7 into one SNARK-attested `newRoot` per batch, lifting the practical mint ceiling from $N=4$ (block-saturated under per-leaf inserts) to $N=256-512$ on L1 with stock Groth16 verifiers and an L2 ceiling of $N=1024+$.

Shipped:

- `circuits/mint_batch.circom` – parameterised batch-mint circuit; `circuits/mint_batch_n{1,2,4,8,16,32}` are the per-N specialisations consumed by `scripts/snark/setup.sh`.
- `src/MintBatchN{1,2,4,8,16,32}Groth16Verifier.sol` – snarkjs-generated per-N verifiers, measured 14.5 KB at $N=16$ and 20.8 KB at $N=32$ (both under the EIP-170 24 KB ceiling).
- `src/MintVerifierAdapter.sol` – now dispatches by `cms.length` to `mintVerifiers[cms.length]`, builds `pub[0..4+N] = [oldRoot, newRoot, nextLeafIndex, totalFace, cm[0..N]]` inline, forwards to the per-N Groth16 verifier.
- `src/Notes.sol` – public inputs include (`oldRoot`, `newRoot`, `nextLeafIndex`, `totalFace`); `filledSubtrees`, `zeros[]`, and `commitmentExists` are retired (duplicate-`cm` migrated to economic self-enforcement via the deterministic nullifier). The 30-slot `roots[]` ring buffer is now updated by writing the SNARK-attested `newRoot` rather than the per-leaf on-chain insert.
- `scripts/snark/prove_mint_batch.js` – new prover script; the off-chain wallet maintains a local mirror of `filledSubtrees` and emits per-leaf sibling paths in submission order.
- `scripts/snark/setup.sh` – runs phase-2 setup per pinned N , exporting one verifier Solidity file per pinned N .
- `test/MintVerifier.t.sol` – 43 tests: full $N=1..16$ ladder (happy path, four tamper paths `totalFace` / `newRoot` / `cm[]` / wrong- N , replay-after-mint, two-batch successive insertion) plus the $N=32$ happy path, cross- N successive ($N=16 \rightarrow N=32$), cross- N replay ($N=4$ proof rejected after live state advances), per- N dispatch coverage, and a partial-batch case (5 live + 11 $v=0$ dummy leaves at $N=16$) confirming dummies advance `nextLeafIndex` by N without inflating `totalFace`.

Cost: full forge measurements across the pinned ladder – $N=1$: 461K, $N=2$: 473K, $N=4$: 500K, $N=8$: 562K, $N=16$: 624K, $N=32$: 1.12M. Per-note cost amortizes from 461K at $N=1$ (the constant $\sim 216K$ Groth16 verifier dominates) to 39K at $N=16$ to 35K at $N=32$, approaching the $\sim 31K$ asymptote implied by the per-leaf calldata + R1CS slope. Spend cost is unchanged at $\sim 360K$ (independent of mint batch size).

Stop conditions for promoting to $N \geq 64$:

- EIP-170 24 KB verifier-bytecode ceiling. $N=64$ lands at ~ 33 KB, breaching the limit. The three escape hatches (calldata-IC hash-pinned, sharded SSTORE2, plain SLOAD) are documented in alberta-buck-notes-rollup-mint.org; calldata-IC is the natural fit for the Holochain wallet.
- Block-share economics. At $N=512$ a single mint occupies $\sim$ half a 30M-gas block; at $N=1024$ it cannot land on L1 at all. L2 deployment (Arbitrum/Base/Optimism, $\sim 100x$ looser block budgets) lifts this; recursion (Nova/Halo2/Plonky3) is the eventual fix above $N=512$.

12.15 Phase 8: A-flavor Spend (spend_a.circom V2 + off-chain CP-DLEQ) – Shipped

Goal: ship the missing half of the spend surface – private redemption of A1/A2 notes that carry an identity binding – with the corrected key-pair-binding semantics surfaced by the 2026-04 Deepseek R4 review. **Both V1 and V2 are now in tree, exercised end-to-end by 26 forge tests in test/SpendAVerifier.t.sol.**

Corrected design invariant (per alberta-buck-proofs.org Theorem 10.1):

- The A-spend identity binding requires a *single* `sk_dep` witness that simultaneously satisfies `pk_dep_current == sk_dep * G` (the spender's registered public key) *and* produces matching decryptions of both the note ciphertext `E_n` and the registered ciphertext `E_reg`. The Chaum-Pedersen DLEQ enforces `C_n - sk_dep * R_n === C_reg - sk_dep * R_reg` – if the recipient lost `sk_rec` and re-registered under a new ElGamal key, no single `sk_dep` satisfies both constraints.
- **Consequence:** A-note loss-recovery via identity re-issuance is *impossible* by design. `sk_rec` must be backed up like a hardware-wallet seed; loss is terminal. The companion documents (alberta-buck-notes.org, alberta-buck-notes-flow.org) were revised in the same cycle to remove the now-incorrect "transparent re-issuance" framing.

Phase 8 ships in two layers, both in tree:

12.15.1 V1: spend_a.circom predicate-only A-spend

The Phase 8 V1 circuit ships the addressed-spend *scaffolding* – A-tag nullifier, flavor constraint, recipient binding – but defers the cryptographic identity gate to V2. The reason is purely engineering: the CP-equality constraint `C_n - sk_dep * R_n === C_reg - sk_dep * R_reg` requires native BN254 G1 arithmetic in-circuit, which is non-native field arithmetic over the BN254 scalar field, costing on the order of 10K-25K constraints per group operation. A full in-circuit CP would multiply spend_a's constraint count by ~10x. Until Phase 8 V2 ships the off-chain verifier (below), A-flavor notes remain predicate-bound only – whoever holds (`rho`, `idHash`, `predicate`) can spend.

12.15.2 V2: In-circuit Poseidon-8 binding + off-chain CP-DLEQ + Solidity precompile verifier

Phase 8 V2 ships the cryptographic identity gate as two cooperating components glued by the new `Notes.spendACP` entry point. The *in-circuit* half is a Poseidon-8 binding gate added to `spend_a.circom V2` that pins the publicly-revealed `E_n` to the leaf's `idHash` at mint time (~265 R1CS). The *off-chain* half is a 4-element Chaum-Pedersen DLEQ proof verified by Solidity using the EIP-196 BN254 precompiles inside `IdentityRegistry.verifySpendCP` (~36K gas). The DLEQ-verification site moves out of the circuit (where it would have cost ~+250K R1CS in non-native BN254 G1 arithmetic) and into `Notes.spendACP`, which calls *both* the SpendA Groth16 verifier *and* `verifySpendCP` atomically. This preserves the V1 invariant (Theorem 10.1): identity re-issuance still cannot recover an A-note, because the second algebraic check fails under any new key, *and* the in-circuit binding stops a spender from substituting a different `E_n` just to make the off-chain check pass with their own key.

The DLEQ statement (single-`sk_dep` variant of Theorem 8 #3):

Prove knowledge of `sk_dep` such that `pk_dep == sk_dep * G = (registered key) (C_reg - C_n) == sk_dep * (R_reg - R_n) =`

Sigma protocol (`alberta_buck.wallet.spend_cp`):

Prover picks `t` in `Fr`, computes `T1 = t * G`, `T2 = t * (R_reg - R_n)`. Challenge: `e = H(R_n, R_reg, C_n, C_reg, pk_dep, T1, T2, recipient, chainid)` Response: `s = t + e * sk_dep` Proof: `(e, s, T1, T2)` – 4 field elements, ~36K gas to verify on-chain.

Architectural note: identity verification location. The A-spend identity check lives in the `Notes.spendACP` API call, not in the SNARK circuit. The spender provides their identity ciphertext `E_n` alongside the spend, and `Notes.spendACP` calls `identityRegistry.verifySpendCP(msg.sender, recipient, E_n, cpProof)` which reads `_E_addr[spender]` and `_pk[spender]` from registry storage. All A1/A2 identity, security, and privacy properties enumerated in `alberta-buck-notes.org` remain in place: only the DLEQ verification *location* moved (off-chain CP via precompiles, instead of non-native G1 in the circuit). The link between the SNARK and the CP verifier is the note's identity ciphertext `E_n`, exposed as 4 additional public inputs to `spend_a.circom` V2 and re-bound to `idHash` via the in-circuit Poseidon-8 binding gate (~265 R1CS). Together the two layers force the spender to (a) prove a leaf opening pinning the published `E_n` to `idHash` at mint time, and (b) prove they own the `sk_dep` that decrypts both `E_n` and the registered `E_reg` to the same identity point `M`.

Why off-chain CP rather than in-circuit:

Option	Cost	Status
In-circuit CP (non-native BN254 G1)	+250K R1CS, +30s prove, +6 KB vkey	Rejected (V1, V2)
BabyJubJub re-key (re-register on Twisted Edwards curve)	Re-architect IdentityRegistry, wallet, all Identity flows	Rejected (scope)
Off-chain CP-DLEQ + EIP-196 precompiles	4 field elts, ~36K gas verify	Selected (V2)

V1 Deliverables (shipped) – predicate-only A-spend:

- `circuits/spend_a.circom` V1 – A-shape redemption circuit. Constraint groups: range bounds on `face/=v`, Poseidon-5 commitment opening, Tornado-style Merkle inclusion (depth 20), A-tag nullifier Poseidon(3)(`rho`, `idHash`, 4243) (4243 domain-separates from B-flavor's 4242), flavor in {A1, A2}, value-conservation ghost bind.
- `src/SpendAGroth16Verifier.sol` – snarkjs-generated verifier for the A-spend circuit.
- `src/Notes.sol` – gained `spendA(...)` entry point (V1; superseded by `spendACP` in V2).
- `scripts/snark/prove_spend_a.js` + `setup.sh` integration.

V2 Deliverables (shipped) – cryptographic identity gate via two cooperating components:

- `alberta_buck/wallet/spend_cp.py` – Python prover and verifier for the CP-DLEQ; mirrors `alberta-buck-proofs.org` Theorem 8 #3 algebra. Tests in `alberta_buck/test/test_spend_cp.py` cover honest spend, identity re-issuance rejection, tampered transcript, and cross-chain/recipient replay.

- `src/IdentityRegistry.sol` – adds `SpendCProof` struct (`e`, `s`, `T1`, `T2`) and `verifySpendCP(spender, recipient, E_n, cpProof)` → bool external view that reads `_E_addr[spender]` and `_pk[spender]` from storage, runs the two algebraic checks via 2 `ecMul` + 2 `ecAdd` EIP-196 precompiles, and recomputes the Fiat-Shamir challenge over the same byte-for-byte transcript order as the Python prover. ~36K gas total.
- `circuits/spend_a.circom` V2 – exposes (`R_n.x`, `R_n.y`, `C_n.x`, `C_n.y`) as 4 new public inputs (9 total) and adds the Poseidon-8 binding constraint `idHash == Poseidon-8(R_n.x, R_n.y, C_n.x, C_n.y, issuerData[0..3])` so the on-chain CP-DLEQ verifier and the SNARK agree on which ciphertext is being spent. `issuerData[4]` is a private witness (A1: `m_issuer` + `sigma`; A2: `E_iss` coords mod `F_R`). This costs ~265 R1CS for one Poseidon-8, well within the existing `pot15` ptau budget. Total ~13,300 R1CS (6152 non-linear – 7140 linear).
- `src/ISpendAVerifier.sol` – new interface distinct from `ISpendVerifier` due to the 9 public inputs (vs B-spend’s 5). Method `verifySpendA(proof, noteRoot, nullifier, face, recipient, chainId, eNoteRx, eNoteRy, eNoteCx, eNoteCy)`.
- `src/SpendAVerifierAdapter.sol` – wraps `ISpendAGroth16Verifier`; decodes proof bytes; packs the 9 public signals in order before calling the underlying `verifyProof`.
- `src/Notes.sol` – gains `spendACP(proof, root, nullifier, face, recipient, E_n, cpProof)`: calls `spendAVerifier.verifySpendA(...)` with `E_n.R/C` coords *and* `identityRegistry.verifySpendCP(m_recipient, E_n, cpProof)`. Both must succeed. Also gains storage slot `identityRegistry` and governance setter `setIdentityRegistry` guarded by `onlyGovernance`, with companion `IdentityRegistryUpdated` event. `spendAVerifier`’s type is now `ISpendAVerifier` (was `ISpendVerifier`).
- `test/SpendAVerifier.t.sol` – 26 forge tests covering happy path, double spend, A-tag nullifier non-collision with B-tag, tamper paths on every SNARK public input (including all four `E_n` coordinates), CP-DLEQ tampers (`e`, `s`, `T1`, `T2`), unregistered-spender rejection, wrong-spender rejection (passing CProof from a different spender), and governance lockdown for both `setSpendAVerifier` and `setIdentityRegistry`. Fixture generated by `scripts/snark/prove_spend_a.js` against vectors from `alberta_buck.wallet.vectors`.

Stop condition: any test that demonstrates an A-note spendable after `sk_rec` loss + identity re-issuance is a hard halt. The cryptographic invariant is the whole point of Phase 8.

12.16 Phase 9: Serialized B1 Sub-Notes (`spend_serialized.circom` + per-parent bitmap)

Goal: amortize bearer-note issuance by anchoring N pseudorandom serials `s_i` under one `cm_parent` leaf and spending each against a shared per-parent nullifier bitmap. The mint circuit is *unchanged* – `cm_parent` is an ordinary Poseidon-5 output – so this phase is spend-side-only on circuits and storage. Per-bearer-note marginal mint cost drops to ~31K/ N_{fam} at $N_{batch}=16$, vs ~31K at the Phase 7-bis baseline.

Per `alberta-buck-notes-serialized.org` migration plan (Phases A-D):

Phase 9-A: Spend-serialized circuit:

- `circuits/spend_serialized.circom` – extend single-note spend with `(subRoot, i, s_i, path_i_in_subRoot)` sub-block; constraint groups are the single-note spend’s plus a depth-K Poseidon-2 sub-tree walk. Constraint count ~25K at K=10 (N_fam=1024), within stock 24 KB verifier.
- `scripts/snark/setup.sh` phase-2 setup (re-uses the existing pot17-pot19 ptau).
- `src/SpendSerializedGroth16Verifier.sol` + adapter route in `SpendVerifierAdapter`.

Phase 9-B: Contract support:

- `Notes.sol` adds `spendSerialized(proof, root, cm_parent, i, face, recipient)` alongside the existing `spend()` and the Phase 8 `spendACP()`.
- New storage mapping `(uint256 => uint256[]) bitmap (parent > packed bitmap)`, allocated on first write per parent; 256 bits per `=uint256` word.
- nullifiers mapping unchanged for single-note and A-flavor flows.

Phase 9-C: Wallet support:

- Mint path: add "serialize with N" option that builds `K_fam, subRoot, predicate_fam, destroys K_fam` after deriving the per-recipient payloads.
- Payload schema v2: extend the bearer-note blob format to include `(i, s_i, path_i_in_subRoot, subRoot)` when the note is serialized.
- Spend path: detect payload variant from schema version; route to the appropriate verifier.
- **Wallet UX requirement:** `K_fam` destruction is the operational trust assumption. The wallet must present this prominently to the issuer and SHOULD attest destruction via HSM if available.

Phase 9-D: Measure + promote:

- Measure spend-serialized gas at N_fam in {16, 256, 1024}. Target: spend-tx ~390-410K (single-note baseline + $\log_2(N_fam)$ sub-tree levels at ~4K each).
- Confirm verifier bytecode size; confirm no EIP-170 breach.
- Document family-linkage disclosure prominently in wallet UX.

Stop condition: any test that demonstrates a sibling sub-note spendable without the recipient’s `(i, s_i, path)` triple, or a `bitmap[cm_parent]` bit settable by anything other than `Notes.spendSerialized` with a valid Groth16 proof.

12.17 Phase 10: Live BuckKController (PID + AMM-pool oracles)

Goal: replace `BuckKControllerStatic` in the Buck constructor with the live `BuckKController` reading commodity-basket prices and the BUCK/USDC TWAP from on-chain sources. The controller is in-tree with mock-feed unit tests; the remaining work is wiring real oracle addresses, picking gains, and validating against historical price paths on a forked-mainnet harness.

Oracle source: AMM pools, not Chainlink (interim):

The architecture document targets Chainlink commodity feeds; the **interim** Phase 10 deployment substitutes Uniswap V3 TWAP reads on USDC-quoted pools for each basket component. Rationale: Chainlink commodity feeds (XAU/USD, AGRI baskets, etc.) are limited in coverage and add an external trust dependency before BUCK has a price discovery surface of its own. AMM-pool oracles – with sufficient TWAP windows and pool depth – are self-sovereign and deployable on any chain without an off-chain oracle integration step.

Initial basket components (illustrative; final composition is governance-set):

Component	Oracle source	Weight	Notes
XAU (gold)	Uniswap V3 PAXG/USDC TWAP	30%	Wrapped-gold proxy
WTI (oil)	Uniswap V3 USOIL-token/USDC	20%	Synthetic if no liquid wrapper
Wheat	Uniswap V3 wheat-token/USDC	10%	Synthetic
Real estate	Tokenised REIT / USDC	20%	RE-tokenised proxy
USD	1.0 (numeraire)	20%	Anchor to the quote currency

Process variable: Uniswap V3 BUCK/USDC TWAP (Phase 11 dependency – pool must exist).

Deliverables:

- `src/BuckKController.sol` – already in-tree with PID structure; complete the `_getBasketCost()` and `_getBuckPrice()` AMM-pool TWAP plumbing. Use `OracleLibrary.consult(pool, twapInterval)` for tick-cumulative deltas; per-pool decimal normalisation helper.
- `script/Deploy.s.sol` – swap `BuckKControllerStatic` -> `BuckKController`, wire the basket-pool addresses + weights, set initial gains (Kp/Ki/Kd), set `dT` and `twapInterval`.
- `test/BuckKController.t.sol` – mocked-pool unit tests (already passing); add fork-test (gated on RPC) that runs the controller against a few thousand blocks of historical price paths and asserts `BUCK_K` trajectories within a reasonable corridor.
- Governance migration runbook: how to flip Buck’s controller pointer atomically.

Stop condition: a fork-test that drives `BUCK_K` outside the `[buckKMin, buckKMax]` band on realistic historical input would indicate gain mistuning or anti-windup logic bug – halt before mainnet promotion.

12.18 Phase 11: BUCK Liquidity Bootstrap + BUCK/USDC AMM Pool

Goal: deploy the BUCK/USDC Uniswap V3 pool that Phase 10 reads as its process variable, seed initial liquidity, and validate the closed loop *BUCK price* -> *BUCK_K* -> *credit limits* -> *BUCK supply* -> *BUCK price*. Without this pool the live controller has no signal.

Deliverables:

- **Treasury minting path:** a one-time governance authorisation to mint a seed quantity of BUCK against a treasury-owned `BUCK_CREDIT` (the seed is collateralised by the same insurance-NFT mechanism every other mint uses; no special path).
- **Pool deployment:** deploy `UniswapV3Pool` BUCK/USDC at an initial fee tier (0.30% suggested for sub-Jubilee bootstrap). Add the BUCK/USDC pool address to `BuckKController` in the same governance batch as the Phase 10 controller swap.

- **Liquidity bootstrap:** provide initial single-sided liquidity (USDC + the treasury-minted BUCK) at a price band tied to the initial BUCK_K target. Documented in `script/BootstrapLiquidity.s.sol`
- **Ongoing test harness:** `test/AMMPool.t.sol` – fork-test that walks several mint -> AMM-swap -> spend -> AMM-swap cycles and asserts that the BUCK_K controller responds with the expected sign and magnitude.
- **Loop validation:** run a multi-day testnet rehearsal with synthetic transaction flow before the controller takes mainnet inputs.

Stop condition: BUCK_K oscillating with growing amplitude under realistic flow indicates gains too aggressive (Phase 10 tuning) or an oracle-update cadence too slow (ΔT too high) – both are halt conditions.

12.19 Phase 12: BUCK_CREDIT Insurance Integration in Buck.mint

Goal: close the loop the architecture document describes – the BUCK_CREDIT NFT is the insurance-backed collateral that bounds BUCK issuance; `Buck.mint()` deduces an instantaneous credit limit from the holder’s deposited NFTs and simultaneously pays the default-insurance premium into the InsurancePool. Most of the mechanics already exist (current `Buck.mint()` reads `buckCredit.totalCurrentValue(sender)` and pays `_computePremium()` to the pool); this phase is integration polish, insurer onboarding, and Jubilee fund hookup.

Deliverables:

- **InsurancePool contract** (`src/InsurancePool.sol`) – ERC-4626 vault that accumulates default-insurance premiums and pays out on documented default events. Pool shares are mutual-insurance equity; depositors absorb claim losses pro-rata. Governance-controlled claim-payout authority for Phase 12; on-chain claim adjudication is a follow-up phase.
- **Default detection:** a periodic (or manually triggerable) view that walks accounts whose `balanceOf(sender) > storedLimit * BUCK_K` – those are in default and need either `burn()` (voluntary), forced liquidation of activated BUCK_CREDIT, or an InsurancePool payout. Off-chain monitoring tooling reads the events; on-chain enforcement is governance in Phase 12 and automated in a follow-up.
- **BUCK_CREDIT insurer onboarding script** (`script/OnboardInsurer.s.sol`) – the mechanical steps to register an insurer, mint a representative BUCK_CREDIT, and validate the full BUCK_CREDIT.mint -> `client.activate` -> `Buck.mint` -> `InsurancePool.deposit` flow.
- **Jubilee fund hookup** – the BUCK demurrage accrual already feeds a single sink whose `_indexAtLastTouch` is weighted-merged on disbursement; Phase 12 routes this sink to a governance-controlled `JubileeFund` contract that periodically distributes accumulated fees to defaulted accounts (or burns them, governance choice).
- **End-to-end mint walkthrough test** (`test/MintWalkthrough.t.sol`) that drives the full Step 0-4 sequence from *Integration: Full mint() and transfer() Walkthrough* above against a forked-mainnet harness with real AMM-pool oracles (Phase 10 + 11) and a representative BUCK_CREDIT.

Stop condition: any path that allows minting BUCK above the credit limit (modulo the documented strict-monotonic `storedLimit` ratchet and the `BUCK_K` reduction default window) is a hard halt. The credit-limit guarantee is the system’s solvency invariant.

12.20 Phase 13: Production MPC Trusted-Setup Ceremony

Goal: replace the dev-only entropy in `scripts/snark/setup.sh` with a production multi-party ceremony for every shipped circuit: `mint_batch_n`{1,2,4,8,16,32} (Phase 7-bis), `spend` (Phase 7), `spend_a` (Phase 8), `spend_serialized` (Phase 9). Each circuit’s `zkey` is circuit-specific; the underlying `ptau` is universal up to the chosen power and can be inherited from a community ceremony (e.g., Perpetual Powers of Tau).

Deliverables:

- **Ceremony plan:** contributor recruitment, attestation ledger, public-coordination thread, per-circuit `zkey` contribution sequence, archival storage for transcripts.
- **Import community ptau:** pull `pot15-pot25` from Perpetual Powers of Tau (or equivalent), verify provenance against the published attestations, mirror to project-controlled storage.
- **Per-circuit phase-2 ceremony:** at least 5 independent contributors per circuit, with the final `zkey` verifiable by any third party against the contribution log.
- **Verifier re-deployment:** regenerate `MintBatchN*Groth16Verifier.sol` and the `spend` verifiers from the production `zkey`; deploy via governance in a single batched transaction (atomic across all circuits to avoid mixed-trust state).
- **Public attestation:** publish ceremony transcripts and verification instructions alongside the deployed contract addresses.

Stop condition: any contributor unable to demonstrate independent entropy generation (e.g., re-using a colleague’s machine, no hardware-RNG attestation) reduces the trust assumption below the "1-of-N honest" threshold. Halt and re-do that contribution before final `zkey` publication.

12.21 Sequencing and Stop Conditions

The phases are strict prerequisites of one another: Phase $n+1$ does not begin until Phase n has a passing CI run and a Foundry gas snapshot. Implementation **STOP** conditions:

- Phase 1: a Solidity `checkChaumPedersen` disagreement with the Python reference vectors. Disagreement implies a bug in the spec or in one of the implementations – resolve before any contract that depends on it ships.
- Phase 2-A: an `Issuer.issue(...)` path that produces a credential whose sigma the wallet’s `rerandomize_for_registration` cannot turn into a sigma’ accepted by `registration_verify` under the issuer’s PS public key. The issuance -> registration handoff is the contract this phase establishes; any drift here propagates into every later identity-bound test.
- Phase 3: `BuckTransferReceipt` hash collisions across distinct ciphertexts (would indicate an ABI-encoding bug – canonicalisation matters).
- Phase 5: any Solidity-side type that does not match the chosen SNARK verifier output (revisit the toolchain choice; do not paper over with adapter contracts).

- Phase 6: a tamper-path test that fails to revert. The adapter + Groth16 verifier must reject every documented attack on `totalFace`, `cm[i]`, batch size, and malformed proof bytes; any false-accept is a hard stop before further Notes work.
- Phase 7: any inconsistency between the mint-circuit commitment layout and the spend circuit's opening layout (would fork the two proof systems' views of a note, silently). Both circuits are pinned at `Poseidon(5)(flavor, v, rho, idHash, predicate)` with `Num2Bits(128)` range bounds on every `v[i]`, `totalFace`, and `face`; the value-conservation constraint `sum(v[i]) == totalFace` now holds in the integers (62-bit margin against field overflow at $N < 2^{64}$). Any future change to either circuit's commitment layout must be made simultaneously in both.
- Phase 7-bis: an `oldRoot` / `newRoot` chain break. The contract guard `oldRoot = roots[currentRootIndex]` and the SNARK derivation of `newRoot` are the two halves of the chain; either side admitting a fabricated `newRoot` silently forks the tree. Any per-N adapter dispatching to the wrong verifier (`cms.length=N` mismatching `mintVerifiers[N]`) is the same class of failure.
- Phase 8: any A-spend test that completes after `sk_rec` loss + identity re-issuance. The single-`sk_dep` witness invariant is the cryptographic substance of the phase; if a compiled circuit accepts unequal `pk_rec_mint` and `pk_dep_current` the constraint set is broken.
- Phase 9: a `bitmap[cm_parent]` bit settable by anything other than `Notes.spendSerialized` with a valid Groth16 proof, OR a sibling sub-note spendable without the recipient's (`i`, `s_i`, `path`) triple. Either is a forge-a-note-I-don't-own admission and a hard halt.
- Phase 10: `BUCK_K` trajectory outside `[buckKMin, buckKMax]` on a fork-test driven by realistic historical price paths. Indicates gain mistuning or anti-windup logic bug; halt before mainnet promotion.
- Phase 11: `BUCK_K` oscillating with growing amplitude under realistic flow. Indicates gains too aggressive (Phase 10) or oracle-update cadence too slow (`dT` too high). Either is a halt condition.
- Phase 12: any path that mints BUCK above the credit limit (modulo the documented strict-monotonic `storedLimit` ratchet and the `BUCK_K` reduction default window). The credit-limit guarantee is the system's solvency invariant.
- Phase 13: any contributor unable to demonstrate independent entropy generation reduces the trust assumption below "1-of-N honest"; halt and re-do that contribution before final `zkey` publication.